JAVA SDE-2 INTERVIEW PREPARATION SERIES

SYSTEM DESIGN AND BACKEND - BOOK 04 OF 14 - STUDY STEP 19E

Spring Framework for Java Engineers

From IoC First Principles to Proxies, Transactions, Web Boundaries, and SDE-2 Incidents

BY

Vinay Reddy Kalluri

A first-principles, executable guide to Spring Framework: container construction, dependency resolution, lifecycle, scopes, AOP, transactions, MVC, async work, testing, and production diagnosis.

SPRING | IOC | PROXIES | TRANSACTIONS | MVC | PRODUCTION

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-29

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `SD 03 — Hibernate and JPA`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Build a precise Java object-graph and proxy mental model before Spring Boot automation, then develop the transaction, web, testing, concurrency, and incident judgment expected from SDE-2 backend engineers.

Readiness map

Decision	Evidence
START HERE	Object construction, candidate selection, lifecycle, or scope is container-managed; A proxy boundary changes AOP, transaction, or async behavior; A transaction, event, web request, executor, or test context has hidden runtime consequences; An SDE-2 interview requires failure and production behavior, not annotation recall.
PREPARE FIRST	Java Foundations Book 01; Maven and Gradle Book 03; Hibernate and JPA Book 03 recommended; Java 21 and a terminal for the executable labs.
MOVE ON WHEN	Construct and diagnose Spring contexts from plain Java through explicit and scanned bean definitions; Predict dependency resolution, lifecycle, scope, configuration mode, event, and validation behavior; Reason exactly about JDK and subclass proxies, self-invocation, advice order, and declarative transactions; Design safe MVC, async, scheduling, testing, outbox, retry, and production boundaries; Answer realistic Spring Framework interview scenarios from object ownership and runtime evidence.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

System Design Segment Position

SD 03 - Hibernate and JPA	YOU ARE HERE SD 04 - Spring Framework	SD 05 - Spring Boot

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Learning Path and Container First Principles

Spring Framework helps an application create objects, connect their dependencies, apply cross-cutting behavior, manage transactions, publish events, validate data, and serve web requests. It does not replace Java. The strongest Spring engineers can first write the object graph in plain Java, then explain exactly which responsibility the container takes over.

This publication targets **Java 21 and Spring Framework 7.0.8**. Spring Framework 7 retains a Java 17 baseline. Version-sensitive features are labeled; the core mental models also apply to supported Spring 6.2 applications.

The one model to keep in your head

TEXT

```
configuration metadata
  @Configuration / @Bean / @Component / XML / programmatic registration
                            |
                            v
                  ApplicationContext
               reads bean definitions
                            |
          +-----------------+-----------------+
          |                 |                 |
          v                 v                 v
      creates           injects          enhances
      objects        dependencies     lifecycle/proxies
          |                 |                 |
          +-----------------+-----------------+
                            |
                            v
                    application beans
```

A **bean** is an object whose creation and lifecycle are managed by a Spring container. It is not a special Java language construct. An `OrderService` remains an ordinary Java class; becoming a bean changes who creates and wires it.

IoC and dependency injection without slogans

Suppose `OrderService` needs an `OrderRepository`.

JAVA

```java
OrderRepository repository = new JdbcOrderRepository(dataSource);
OrderService service = new OrderService(repository);
```

The application entry point controls construction. With dependency injection, `OrderService` still declares what it needs, but another component supplies it:

JAVA

```java
final class OrderService {
    private final OrderRepository repository;

    OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}
```

Spring can become that object-graph assembler. This is **inversion of control**: creation and connection move from business objects to a container. DI is the concrete mechanism: dependencies arrive through constructors, factory methods, or properties instead of being looked up or created inside the object.

Why this matters

- Dependencies become visible in the class API.
- Tests can supply small fakes without starting Spring.
- Configuration can choose an implementation at an application boundary.
- Framework services such as transactions can wrap calls consistently.

DI does not automatically create good design. A class with fifteen constructor arguments is still telling you that its responsibility may be too large.

Spring Framework versus Spring Boot

Spring Framework	Spring Boot
Container, DI, AOP, transactions, events, MVC, validation, testing	Opinionated application setup built on Framework
You can construct a context explicitly	Usually creates and configures the context for you
Teaches the mechanisms	Adds auto-configuration, starters, executable packaging, and operations conventions

This book does not use Boot to hide Framework mechanics. Boot is covered in **SD 05 - Spring Boot** after this volume.

Three boundaries to label in interviews

1. **Java boundary:** constructors, interfaces, objects, threads, exceptions.
2. **Spring container boundary:** bean definitions, dependency resolution, lifecycle, proxies.
3. **External resource boundary:** database transaction, network call, message broker, filesystem.

An annotation is metadata. It has an effect only when a configured Spring component discovers it and the call crosses the required runtime boundary. For example, `@Transactional` normally needs transaction infrastructure and a call through a Spring proxy.

The learning sequence

TEXT

```
plain Java object graph
        |
        v
first context -> bean definitions -> injection -> configuration data
        |
        v
lifecycle -> scopes -> configuration mechanics -> events/validation
        |
        v
AOP -> proxies -> transactions -> web flow -> async work
        |
        v
testing -> production diagnosis -> realistic interviews
```

Do not jump to transaction propagation before you can identify the proxy. Do not add a custom post-processor before you understand the normal lifecycle. Do not start a full application context when a pure Java unit test proves the behavior.

The running domain

Examples use a small order application:

TEXT

```
OrderController -> OrderService -> OrderRepository -> database
                        |
                        +-> PaymentGateway (remote system)
                        +-> application event
```

The business invariant is simple: one request key creates at most one order. The design questions are not simple: which dependencies are beans, where does the transaction begin, when should an event be observed, and what happens when a remote call fails?

A six-question runtime trace

For every Spring example, ask:

1. Which class registered the bean definition?
2. Which container owns the bean instance?
3. How was each dependency selected?
4. Did the caller receive a target or a proxy?
5. Which thread and transaction execute the method?
6. How will a test or metric prove the answer?

Common myths corrected

- Spring is not the same as Spring Boot.
- `@Autowired` does not make a dependency globally available; the container resolves an injection point.
- Spring singleton means one instance per bean definition per container, not one object per JVM.
- A proxy cannot intercept every possible Java call.
- `@Transactional` does not create a distributed transaction across remote services.
- Constructor injection improves visibility, but it does not repair circular or oversized designs.

Quick check

1. What makes an ordinary Java object a Spring bean?
2. How is DI a form of IoC?
3. Why can an annotation be present but ineffective?
4. What is the difference between Framework and Boot?
5. What six questions should you ask when tracing runtime behavior?

Practice

- **Foundation:** Draw a three-object graph in plain Java and mark who constructs each object.
- **Foundation:** Rewrite a class that calls `new JdbcOrderRepository()` internally to use constructor injection.
- **Interview Core:** Explain why a bean with twelve dependencies may indicate a design problem.
- **Interview Core:** Classify configuration, proxying, and database commit by boundary.
- **SDE-2 Follow-up:** Given an ineffective annotation, describe the evidence you would collect before changing configuration.

TRY IT | Readiness checkpoint

Continue when you can define bean, IoC, and DI without saying "Spring magic," and can separate a Java object, its container registration, and any proxy surrounding it.

SDE-2 CORE CHAPTER

Chapter 2 - Plain Java to the First Application Context

Start with working plain Java. Spring should remove assembly repetition, not hide an object model you never understood.

A plain Java object graph

JAVA

```java
interface MessageSender {
    void send(String recipient, String text);
}

final class ConsoleMessageSender implements MessageSender {
    @Override
    public void send(String recipient, String text) {
        System.out.println(recipient + ": " + text);
    }
}

final class WelcomeService {
    private final MessageSender sender;

    WelcomeService(MessageSender sender) {
        this.sender = sender;
    }

    void welcome(String email) {
        sender.send(email, "Welcome");
    }
}

public class PlainJavaApplication {
    public static void main(String[] args) {
        MessageSender sender = new ConsoleMessageSender();
        WelcomeService service = new WelcomeService(sender);
        service.welcome("reader@example.com");
    }
}
```

Expected output:

TEXT

```
reader@example.com: Welcome
```

`WelcomeService` depends on the interface, not on Spring. Its constructor is a complete statement of required collaborators.

Add only the container

The minimal Maven dependency is `spring-context`; it brings the core container modules transitively. Pin the version through your build rather than copying a floating version from a blog.

XML

```xml
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>7.0.8</version>
</dependency>
```

Register the same graph through Java configuration:

JAVA

```java
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
class ApplicationConfiguration {
    @Bean
    MessageSender messageSender() {
        return new ConsoleMessageSender();
    }

    @Bean
    WelcomeService welcomeService(MessageSender sender) {
        return new WelcomeService(sender);
    }
}

public class SpringApplication {
    public static void main(String[] args) {
        try (var context = new AnnotationConfigApplicationContext(
                ApplicationConfiguration.class)) {
            WelcomeService service = context.getBean(WelcomeService.class);
            service.welcome("reader@example.com");
        }
    }
}
```

Expected output is unchanged. That is a feature: business behavior did not become a framework concern.

TRACE IT | Dry run: what the context does

1. Read `ApplicationConfiguration` as configuration metadata.
2. Register bean definitions for `messageSender` and `welcomeService`.
3. Create the `MessageSender` singleton.
4. Resolve the `MessageSender` parameter of `welcomeService`.
5. Invoke the factory method and store the resulting singleton.
6. Publish context lifecycle events and make the context ready.
7. Return the existing `WelcomeService` when `getBean` is called.
8. On `close`, run eligible destruction callbacks.

Calling `getBean` is useful at the composition root and in demonstrations. Business classes should usually receive dependencies instead of pulling them from the context. Frequent service-locator calls hide dependencies and couple domain code to Spring.

BeanFactory versus ApplicationContext

`BeanFactory` is the foundational bean-container contract. `ApplicationContext` extends it and adds application-oriented capabilities such as event publication, resource loading, message resolution, environment access, and automatic detection of many extension beans.

Question	Preferred answer
Which one do applications normally use?	`ApplicationContext`
Is `BeanFactory` obsolete?	No; it is the lower-level container contract
Does `ApplicationContext` eagerly create every bean?	It pre-instantiates non-lazy singleton beans by default, with exceptions for lazy and other scopes
Should business code depend on either?	Usually no

Startup failure is valuable

Constructor injection allows missing or ambiguous dependencies to fail during context refresh rather than during a production request.

JAVA

```java
@Bean
WelcomeService welcomeService(MessageSender sender) {
    return new WelcomeService(sender);
}
// No MessageSender bean -> context refresh fails.
```

Treat startup validation as an operational control. A service that cannot construct its required object graph should fail before receiving traffic.

Debugging a first context

When startup fails, read from the deepest cause outward:

1. Find `NoSuchBeanDefinitionException`, `NoUniqueBeanDefinitionException`, construction failure, or configuration error.
2. Identify the exact injection point and required type/name/qualifier.
3. List candidate bean definitions, including profiles and imports.
4. Repair the object graph; do not add unrelated component scans.

WATCH OUT | Common mistakes

- Creating a second context accidentally and expecting its beans to be shared.
- Calling `new WelcomeService(...)` and expecting Spring advice on that unmanaged object.
- Hiding all construction behind static access.
- Closing a context that another part of the process still owns.
- Starting Spring in every test when a constructor call is enough.

Interview angle

Interviewer: What happens during `ApplicationContext` startup?

Strong answer: Spring reads configuration into bean definitions, runs definition-level extension points, instantiates eligible singletons, resolves dependencies, invokes aware and lifecycle callbacks through registered post-processors, may wrap beans in proxies, validates the graph, and publishes refresh lifecycle events. I distinguish definition metadata from actual instances and mention that lazy or non-singleton beans may be created later.

Quick check

1. Why does the first Spring example preserve ordinary Java constructors?
2. What additional capabilities does `ApplicationContext` provide?
3. When are normal non-lazy singletons created?
4. Why is `getBean` inside business logic usually a smell?
5. Why is a startup wiring failure safer than a late request failure?

Predict and debug

Predict: Two calls to `context.getBean(WelcomeService.class)` return what by default? The same singleton instance for that bean definition.

Debug: A developer creates `new WelcomeService(sender)` and reports that an aspect never runs. The object was not obtained through the container, so no Spring proxy surrounds it.

Practice

- **Foundation:** Convert a plain `ReportService -> Clock` graph to two `@Bean` methods.
- **Foundation:** Print whether two lookups return the same reference.
- **Interview Core:** Explain context refresh in eight steps without using Boot terminology.
- **Interview Core:** Write a unit test for `WelcomeService` with a fake sender and no Spring.
- **SDE-2 Follow-up:** Design startup checks for three required external configuration values without connecting to a production system.

TRY IT | Readiness checkpoint

Continue when you can build and close an `AnnotationConfigApplicationContext`, distinguish definition from instance, and explain why business classes remain testable without Spring.

SDE-2 CORE CHAPTER

Chapter 3 - Bean Definitions, Java Configuration, and Component Scanning

The container works from **bean definitions**: recipes describing how an object is created, named, scoped, initialized, and connected. An annotation is one way to contribute a recipe; it is not the bean itself.

Three registration styles

Explicit Java configuration

JAVA

```java
@Configuration
class OrderConfiguration {
    @Bean
    Clock clock() {
        return Clock.systemUTC();
    }

    @Bean
    OrderService orderService(OrderRepository repository, Clock clock) {
        return new OrderService(repository, clock);
    }
}
```

Advantages: construction is visible, third-party classes are easy to register, and a configuration class forms a reviewable module boundary.

Component scanning

JAVA

```java
@Repository
final class JdbcOrderRepository implements OrderRepository {
    // ...
}

@Service
final class OrderService {
    private final OrderRepository repository;

    OrderService(OrderRepository repository) {
        this.repository = repository;
    }
}

@Configuration
@ComponentScan(basePackageClasses = OrderService.class)
class ApplicationConfiguration {
}
```

`@Component`, `@Service`, `@Repository`, and `@Controller` are stereotype annotations. The specialized forms communicate architectural role. `@Repository` also participates in persistence-exception translation when the relevant infrastructure is configured.

XML or programmatic registration

Legacy and integration-heavy systems may use XML. Framework libraries may register definitions through lower-level APIs. SDE-2 engineers should be able to read XML, but new examples in this book prefer explicit Java configuration and focused scanning.

Scanning is a search boundary

Avoid a scan rooted at a broad company package merely to make startup pass. It can register tests, duplicate adapters, or internal configuration unintentionally.

JAVA

```java
@ComponentScan(basePackages = "com.example") // very broad
```

Prefer a marker type or a narrow package:

JAVA

```java
@ComponentScan(basePackageClasses = OrdersModule.class)
```

This survives package renames better than a string and states the intended module.

Bean names and aliases

An `@Bean` method defaults to its method name. A scanned component defaults to a decapitalized class-based name unless explicitly named.

JAVA

```java
@Bean("primaryClock")
Clock applicationClock() {
    return Clock.systemUTC();
}
```

Type-based injection should not depend on incidental names. Use explicit names or qualifiers when multiple semantic implementations exist.

@Import creates visible modules

JAVA

```java
@Configuration
@Import({OrderConfiguration.class, MessagingConfiguration.class})
class RootConfiguration {
}
```

`@Import` is often easier to reason about than a large scan. A useful rule is: use scanning for stable application components inside a bounded package; use explicit imports and `@Bean` methods at infrastructure and module boundaries.

Third-party objects

You cannot annotate a class you do not own, but you can still manage it:

JAVA

```java
@Bean
PaymentClient paymentClient(PaymentClientSettings settings) {
    return new PaymentClient(settings.baseUrl(), settings.timeout());
}
```

The factory method is also the right place to translate validated configuration into a library-specific object.

Definition versus instance

TEXT

```
BeanDefinition "orderService"
  class/factory: OrderConfiguration.orderService(...)
  scope: singleton
  lazy: false
  dependencies: OrderRepository, Clock
                  |
                  v refresh
OrderService instance (possibly wrapped by a proxy)
```

Definition-level tools run before most application beans exist. Instance-level tools run around construction and initialization. Confusing these stages causes many extension-point bugs.

Duplicate registration and overriding

If two configurations define the same name, behavior depends on the context and override policy. Do not rely on whichever definition happens to win. Give semantically distinct beans distinct names or eliminate the duplicate. A production graph should be deterministic under package ordering changes.

Configuration design example

Weak:

JAVA

```java
@Configuration
@ComponentScan("com.company")
class EverythingConfiguration {
}
```

Stronger:

JAVA

```java
@Configuration
@Import({OrdersConfiguration.class, PaymentsConfiguration.class})
class BackendConfiguration {
}

@Configuration
@ComponentScan(basePackageClasses = OrdersModule.class)
class OrdersConfiguration {
    @Bean
    Clock orderClock() {
        return Clock.systemUTC();
    }
}
```

The stronger design exposes application modules and keeps infrastructure decisions reviewable.

WATCH OUT | Common mistakes

- Believing `@Service` changes Java call semantics by itself.
- Scanning the default package or the whole classpath.
- Registering one component both through scanning and an `@Bean` method.
- Using bean names as hidden business routing.
- Placing environment-specific branching throughout business configuration.
- Assuming imported classes move into the current Java package.

Interview angle

Interviewer: Component scanning or explicit `@Bean` methods?

Strong answer: Both produce bean definitions. I use focused scanning for application-owned components with stable stereotypes and explicit configuration for third-party clients, environment-bound objects, and module assembly. I avoid broad scans because they hide registration and increase startup surprises. The decision is about graph visibility and ownership, not annotation preference.

Quick check

1. What information belongs in a bean definition?
2. How does an `@Bean` name default?
3. Why can a marker class make scanning safer?
4. When is `@Bean` preferable to `@Component`?
5. What is the difference between definition-level and instance-level processing?

Predict and debug

Predict: If `OrderService` is annotated but its package is not scanned or imported, is it a bean? No.

Debug: Startup finds two `PaymentGateway` beans. Do not widen scanning further. Name the intended semantics, use a qualifier or primary only when one default is real, and make the configuration boundary explicit.

Practice

- **Foundation:** Register `Clock`, `TaxPolicy`, and `CheckoutService` with `@Bean`.
- **Foundation:** Convert only application-owned services to scanned components.
- **Interview Core:** Draw definitions and instances for a three-bean context.
- **Interview Core:** Find the duplicate-registration risk in a scan-plus-import configuration.
- **SDE-2 Follow-up:** Split a 40-bean configuration into reviewable modules without creating child contexts.

TRY IT | Readiness checkpoint

Continue when you can locate the exact registration path for every bean and defend the scan or explicit configuration boundary.

SDE-2 CORE CHAPTER

Chapter 4 - Constructor Injection and Dependency Resolution

Dependency injection is easy when exactly one bean matches. Interviews become interesting when dependencies are optional, repeated, ordered, lazy, or circular.

Constructor injection as the default

JAVA

```java
@Service
final class CheckoutService {
    private final PriceCalculator calculator;
    private final PaymentGateway gateway;

    CheckoutService(PriceCalculator calculator, PaymentGateway gateway) {
        this.calculator = Objects.requireNonNull(calculator);
        this.gateway = Objects.requireNonNull(gateway);
    }
}
```

For a single constructor, `@Autowired` is unnecessary. Constructor injection makes required dependencies visible, supports final fields, and makes plain unit construction natural.

Setter injection can express a genuinely optional or reconfigurable collaborator. Field injection hides construction requirements, prevents final fields, and makes non-container tests awkward; avoid it in application code.

Resolution by type, then disambiguation

Given two implementations:

JAVA

```java
@Component
final class StripeGateway implements PaymentGateway { }

@Component
final class OfflineGateway implements PaymentGateway { }
```

an unqualified `PaymentGateway` injection is ambiguous. Options:

One real default with @Primary

JAVA

```java
@Primary
@Component
final class StripeGateway implements PaymentGateway { }
```

Use `@Primary` only if one implementation is the application-wide default.

Semantic selection with @Qualifier

JAVA

```
CheckoutService(@Qualifier("offlineGateway") PaymentGateway gateway) {
    this.gateway = gateway;
}
```

For durable semantics, define a custom qualifier instead of repeating strings:

JAVA

```
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Qualifier
@interface OfflinePayments { }
```

Inject all strategies

JAVA

```
FraudRouter(List<FraudRule> rules) {
    this.rules = List.copyOf(rules);
}
```

Spring supplies all matching beans. Use `@Order` or `Ordered` only when order is part of the contract; otherwise do not let registration order become behavior.

Optional and deferred dependencies

JAVA

```
final class ExportService {
    private final ObjectProvider<AuditSink> auditSink;

    ExportService(ObjectProvider<AuditSink> auditSink) {
        this.auditSink = auditSink;
    }

    void export() {
        auditSink.ifAvailable(sink -> sink.record("export"));
    }
}
```

`ObjectProvider<T>` supports optional, lazy, or repeated resolution while keeping the dependency explicit. `Optional<T>` can represent simple absence. Do not make a required business collaborator optional merely to keep startup green.

Generic types participate in matching

JAVA

```java
interface Handler<T> {
    void handle(T command);
}

final class CreateOrderHandler implements Handler<CreateOrder> { }
```

Spring can use generic type information when resolving candidates. This makes typed strategy registries possible, but keep routing explicit and test ambiguous cases.

Parameter names are not your primary contract

Spring may use an injection-point name as a fallback among candidates when parameter metadata is available. That is fragile as the main design. Prefer type, semantic qualifiers, or a deliberate collection/map.

Circular dependencies are a design signal

TEXT

```
OrderService -> PricingService -> OrderService
```

Constructor injection makes the cycle fail clearly. Field/setter injection may permit some cycles through early references in some configurations, but the graph remains hard to construct, test, and reason about. Typical corrections:

- extract the shared policy into a third service;
- publish a domain/application event when coupling is temporal;
- move orchestration into a higher-level coordinator;
- merge classes only if they are truly one responsibility.

`@Lazy` can defer one side and break construction, but it often conceals a design issue. Use it only after naming why the cycle is legitimate.

Resolution decision tree

TEXT

```
one injection point of type T
        |
        v
find candidate beans assignable to T
        |
        +-- none -> optional/provider? otherwise fail
        |
        +-- one -> inject it
        |
        +-- many -> qualifier/primary/fallback/name rules
                        |
                        +-- still ambiguous -> fail startup
```

Spring Framework 7 also documents `@Fallback` for candidates that should lose to regular beans. Treat it as version-sensitive; `@Primary` and explicit qualifiers remain common interview expectations.

WATCH OUT | Common mistakes

- Using field injection because it is shorter.
- Adding `@Primary` to silence ambiguity without defining a real default.
- Injecting a `Map<String, Strategy>` and leaking bean names into domain behavior accidentally.
- Making every dependency optional.
- Applying `@Lazy` to hide a cycle without redesigning responsibilities.
- Expecting constructor injection to prevent mutable internal state.

Interview angle

Interviewer: Why do you prefer constructor injection?

Strong answer: It makes required dependencies part of the type's construction contract, supports final references, fails an incomplete graph at startup, and allows pure Java tests. I do not claim it makes objects immutable or fixes too many responsibilities. Optional collaborators are modeled explicitly, and multiple candidates are selected by semantic qualifier or a deliberate strategy collection.

Quick check

1. When is `@Autowired` optional on a constructor?
2. What is the difference between `@Primary` and `@Qualifier`?
3. When is `ObjectProvider` appropriate?
4. Why is collection ordering a contract decision?
5. What does a constructor cycle reveal?

Predict and debug

Predict: Two `PaymentGateway` beans, neither qualified or primary, and one `PaymentGateway` parameter cause context refresh to fail with an ambiguity.

Debug: A developer changes the parameter name to match a bean and the error disappears. Replace the incidental name match with an explicit semantic qualifier.

Practice

- **Foundation:** Refactor a field-injected service into constructor injection.
- **Foundation:** Register two `Clock` beans and select one with a qualifier.
- **Interview Core:** Inject and order three validation rules; write a test for their sequence.
- **Interview Core:** Model an optional audit sink without returning null.
- **SDE-2 Follow-up:** Redesign an `OrderService <-> InventoryService` cycle and defend the new boundary.

TRY IT | Readiness checkpoint

Continue when you can predict zero, one, or many-candidate outcomes and can remove a circular dependency without hiding it behind lazy injection.

SDE-2 CORE CHAPTER

Chapter 5 - Environment, Properties, Profiles, and Resources

Configuration is input to the object graph. Treat it as typed, validated data with explicit ownership, not as strings fetched throughout business code.

Environment and property sources

The Spring `Environment` represents profiles and property resolution. Property sources can include JVM system properties, environment variables, and explicitly registered files or maps. Precedence matters: the first matching property source wins according to the configured order.

JAVA

```java
@Bean
PaymentClient paymentClient(Environment environment) {
    String baseUrl = environment.getRequiredProperty("payments.base-url");
    Duration timeout = environment.getRequiredProperty(
            "payments.timeout", Duration.class);
    return new PaymentClient(baseUrl, timeout);
}
```

This is reasonable at a configuration boundary. Passing `Environment` into domain services lets arbitrary keys spread through the codebase and makes required input invisible.

Property-token injection

JAVA

```java
final class PaymentSettings {
    private final Duration timeout;

    PaymentSettings(@Value("${payments.timeout:2s}") Duration timeout) {
        this.timeout = timeout;
    }
}
```

`@Value` can resolve property tokens and expressions when the appropriate infrastructure is registered. It is convenient for a few values. For a larger related set, build a typed settings object in configuration, validate it once, and inject that object.

Precedence must be tested

TEXT

```
high priority  test override
               command/JVM property
               environment-derived source
               application file
low priority   code default
```

This diagram is illustrative, not a universal promise. Spring Boot defines extensive external-configuration precedence; plain Framework applications define the property sources they add. State which environment you mean.

Profiles select graphs, not business branches

JAVA

```java
@Configuration
@Profile("local")
class LocalPaymentConfiguration {
    @Bean
    PaymentGateway paymentGateway() {
        return new StubPaymentGateway();
    }
}
```

Profiles are useful for coarse environment or deployment variants. Avoid a combinatorial set such as `local`, `cloud`, `eu`, `async`, `debug`, `new-checkout`. Prefer explicit configuration objects and feature-decision services for independent runtime choices.

Profile expressions can combine conditions, but complexity is still complexity. A graph that is impossible to visualize is difficult to validate before deployment.

Resource abstracts location

JAVA

```java
@Bean
TermsService termsService(ApplicationContext context) {
    Resource resource = context.getResource("classpath:terms/default.txt");
    return new TermsService(resource);
}
```

Common prefixes include `classpath:` and `file:`. A `Resource` may not be a real filesystem `File`; classpath resources inside a JAR are a classic example. Use `getInputStream()` when streaming is the actual requirement.

JAVA

```java
try (InputStream input = resource.getInputStream()) {
    // read with an explicit charset and bounded size
}
```

Never assume a resource is repeatable or small. Close streams and apply size limits to untrusted content.

Message resolution

`ApplicationContext` also implements `MessageSource`, supporting codes, arguments, and locales:

JAVA

```java
String message = context.getMessage(
        "order.not-found", new Object[]{orderId}, Locale.US);
```

Business errors should carry stable codes and structured context. Human-language messages can be resolved at the presentation boundary.

Secrets and operational safety

- Do not commit secrets to property files.
- Keep secret values out of exception messages and configuration dumps.
- Validate presence and format during startup, but avoid contacting every remote service in bean constructors.
- Document which settings are reloadable; plain bean construction usually captures a startup snapshot.
- Record non-secret effective configuration and active profiles for diagnosis.

Strong configuration boundary

JAVA

```java
record PaymentClientSettings(URI baseUri, Duration timeout) {
    PaymentClientSettings {
        Objects.requireNonNull(baseUri);
        Objects.requireNonNull(timeout);
        if (!baseUri.getScheme().equals("https")) {
            throw new IllegalArgumentException("payments URL must use HTTPS");
        }
        if (timeout.isZero() || timeout.isNegative()) {
            throw new IllegalArgumentException("timeout must be positive");
        }
    }
}
```

Construct this once from external strings. Downstream code receives typed, already-validated values.

WATCH OUT | Common mistakes

- Assuming Spring Framework and Spring Boot have identical property precedence.
- Reading configuration independently in many beans.
- Treating profile names as runtime business states.
- Calling `resource.getFile()` for a resource packaged in a JAR.
- Providing insecure production defaults for required secrets or endpoints.
- Logging tokens while diagnosing a missing property.

Interview angle

Interviewer: How would you manage environment-specific clients without scattering conditionals?

Strong answer: I keep business services dependent on a stable interface. At configuration startup, I resolve and validate typed settings, then choose and construct one adapter through explicit configuration or a coarse profile. I test property precedence and graph selection, fail fast for invalid required values, and never expose secrets in diagnostics.

Quick check

1. What two concerns does `Environment` model?
2. Why should property precedence be an explicit test?
3. When do profiles become difficult to manage?
4. Why may `Resource#getFile()` fail?
5. Where should message localization occur?

Predict and debug

Predict: A `classpath:` resource inside an executable JAR can provide an input stream but may not have a usable filesystem path.

Debug: `payments.timeout=abc` reaches a request and fails late. Convert and validate it when building `PaymentClientSettings`, causing context startup to fail with the property name and safe reason.

Practice

- **Foundation:** Load one required string and one `Duration` from `Environment`.
- **Foundation:** Read a classpath text resource without converting it to `File`.
- **Interview Core:** Design a typed settings record for a database client.
- **Interview Core:** Test an override property against a default property source.
- **SDE-2 Follow-up:** Replace eight interacting profiles with two deployment profiles and explicit feature configuration.

TRY IT | Readiness checkpoint

Continue when you can turn external strings into a small validated configuration object and explain resource and profile boundaries without borrowing unmentioned Boot behavior.

SDE-2 CORE CHAPTER

Chapter 6 - Bean Lifecycle and Container Extension Points

The bean lifecycle is a pipeline. Knowing the order explains why a dependency is null, why a callback ran twice, or why one bean missed proxying.

Lifecycle of a normal singleton

TEXT

```
bean definition registered
        |
        v
instantiate object
        |
        v
populate dependencies and properties
        |
        v
Aware callbacks (when implemented)
        |
        v
BeanPostProcessor before initialization
        |
        v
initialization callbacks
        |
        v
BeanPostProcessor after initialization
        |       (a proxy may be returned here)
        v
ready bean exposed to callers
        |
        v
context close -> destruction callbacks for eligible scopes
```

This is a learning model, not every internal callback. The key distinction is between changing **bean definitions before instances exist** and processing **bean instances around initialization**.

Initialization and destruction

Prefer a plain method named in configuration when practical:

JAVA

```java
final class ConnectionRegistry {
    void initialize() {
        // Validate local state; do not perform unbounded remote work.
    }

    void close() {
        // Release resources owned by this bean.
    }
}

@Bean(initMethod = "initialize", destroyMethod = "close")
ConnectionRegistry connectionRegistry() {
    return new ConnectionRegistry();
}
```

`@PostConstruct` and `@PreDestroy` are also common and use `jakarta.annotation` in current Jakarta-based applications. `InitializingBean` and `DisposableBean` work but couple the class to Spring. Pick one lifecycle style for a component; combining all styles creates order questions and duplicate work.

Constructors should establish local validity

A constructor should validate required arguments and establish invariants. Avoid long network calls, thread creation, or database migrations in it. Heavy startup work:

- increases failure ambiguity;
- runs before the full context is ready;
- complicates tests and shutdown;
- may be repeated during test context creation.

If the application must not accept traffic until a dependency is ready, use an explicit readiness component with bounded timeouts and observable failure rather than a hidden constructor call.

Definition-level extension

`BeanFactoryPostProcessor` can inspect or change bean definitions before ordinary beans are instantiated. A familiar implementation resolves external property tokens. Custom implementations are infrastructure code and should avoid creating application beans prematurely.

JAVA

```java
@Bean
static BeanFactoryPostProcessor requireNamedBean() {
    return factory -> {
        if (!factory.containsBeanDefinition("orderService")) {
            throw new IllegalStateException("orderService definition is required");
        }
    };
}
```

The factory method is `static` so the post-processor can be registered without early instantiation of its configuration class.

Instance-level extension

`BeanPostProcessor` sees bean instances before and after initialization. Framework features use post-processors for annotation handling and proxy creation.

JAVA

```java
final class TimingMarkerPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String name) {
        if (bean instanceof TimedComponent) {
            System.out.println("timed bean: " + name);
        }
        return bean;
    }
}
```

Returning a different object is legal; that is one route to a proxy. A post-processor must return the bean, even when unchanged. Returning null prematurely stops subsequent processing for that phase.

Aware callbacks and framework coupling

Interfaces such as `BeanNameAware` and `ApplicationContextAware` inject container infrastructure. They are appropriate in framework adapters but are usually a smell in business services. If a service needs a clock, publisher, or strategy, inject that focused interface rather than the entire context.

Lifecycle ordering

`@DependsOn` can force initialization order, but dependency injection is clearer when one bean truly depends on another. `SmartLifecycle` coordinates start/stop phases for active components such as listeners. It is not a substitute for a well-defined dependency graph.

On shutdown, give in-flight work a bounded drain period, stop accepting new work, and release resources. Do not assume a destruction callback runs after abrupt process termination.

Prototype lifecycle limit

Spring creates and initializes prototype beans but does not automatically call their destruction callbacks. The recipient owns cleanup. This is one reason prototype beans should not casually own files, threads, or connections.

WATCH OUT | Common mistakes

- Calling application beans from a factory post-processor and causing early creation.
- Returning null from a post-processor accidentally.
- Starting non-daemon threads without a shutdown contract.
- Expecting destruction callbacks after `kill -9` or a crash.
- Implementing `ApplicationContextAware` as a service locator.
- Registering a post-processor with a factory method whose return type hides its post-processor nature.

Interview angle

Interviewer: Difference between `BeanFactoryPostProcessor` and `BeanPostProcessor`?

Strong answer: The first works on bean-definition metadata before normal instances are created. The second works on instances around initialization and may return wrappers such as proxies. I avoid resolving application beans from either too early because that can skip later processing or create incomplete infrastructure.

Quick check

1. At what stage are dependencies populated?
2. Why may the final exposed bean differ from the constructed target?
3. Which extension point changes definition metadata?
4. Who destroys prototype instances?
5. Why should business code avoid `ApplicationContextAware`?

Predict and debug

Predict: An `@Bean` method returns a class with `close()`. Spring may infer and call a public close/shutdown destroy method for a singleton, but explicit lifecycle configuration is clearer for interview reasoning.

Debug: A bean is not advised because it was fetched while an infrastructure post-processor was being constructed. Inspect early creation warnings, simplify the post-processor dependencies, and ensure infrastructure return types are declared precisely.

Practice

- **Foundation:** Add init and destroy methods to a managed resource and close the context.
- **Foundation:** Write the lifecycle stages in order from definition to shutdown.
- **Interview Core:** Build a post-processor that records bean names without changing them.
- **Interview Core:** Explain why a slow constructor damages startup diagnosis.
- **SDE-2 Follow-up:** Design start, readiness, drain, stop, and forced-termination behavior for a message listener.

TRY IT | Readiness checkpoint

Continue when you can locate an action at definition time, instance initialization, post-processing, runtime, or shutdown and name who owns cleanup.

SDE-2 CORE CHAPTER

Chapter 7 - Bean Scopes, State, and Thread Safety

Scope answers: **for one bean definition, when is a new instance supplied and who owns its lifecycle?** It does not automatically make a class thread-safe.

Core scopes

Scope	Instance boundary	Lifecycle note
`singleton`	one instance per bean definition per container	full lifecycle managed on normal context close
`prototype`	a new instance when the container resolves or requests it	destruction is not managed automatically
`request`	one instance per HTTP request	web-aware context required
`session`	one instance per HTTP session	web-aware context required; distributed session concerns remain
`application`	one per `ServletContext`	not identical to Spring singleton semantics
`websocket`	one per WebSocket session	web context required

Spring singleton is not the GoF singleton pattern and not one per JVM. Two contexts can each own a singleton from the same definition class. Two bean definitions can each create one instance of the same Java class.

Stateless singleton as the default

JAVA

```java
@Service
final class PriceService {
    Money calculate(Cart cart, PricingPolicy policy) {
        return policy.price(cart);
    }
}
```

Method-local values are naturally isolated between calls. By contrast:

JAVA

```java
@Service
final class UnsafeSequenceService {
    private long next = 1;

    long nextValue() {
        return next++;
    }
}
```

Concurrent calls race. Singleton scope did not serialize access. Use a database sequence, atomic primitive when semantics fit, or another coordination design. Never store request-specific user data in singleton fields.

Prototype injection surprise

JAVA

```java
@Component
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
final class WorkBuffer { }

@Service
final class ExportService {
    private final WorkBuffer buffer;

    ExportService(WorkBuffer buffer) {
        this.buffer = buffer;
    }
}
```

`ExportService` is a singleton, so its constructor runs once. It receives one prototype at that time and reuses it. Prototype means new on resolution, not automatically new on every method call.

For a fresh instance per operation:

JAVA

```java
final class ExportService {
    private final ObjectProvider<WorkBuffer> buffers;

    ExportService(ObjectProvider<WorkBuffer> buffers) {
        this.buffers = buffers;
    }

    void export() {
        WorkBuffer buffer = buffers.getObject();
        // use and explicitly clean up owned resources
    }
}
```

Often a plain `new WorkBuffer()` in application code is simpler if the buffer has no container-provided dependencies.

Scoped proxies

A singleton controller/service cannot hold the actual request-scoped object at startup because no request exists. A scoped proxy can be injected instead:

TEXT

```
singleton service -> stable request-scope proxy
                              |
                     each call resolves target
                              |
                              v
                     current request instance
```

The proxy does not copy request data into other threads. An `@Async` task has no automatic guarantee that request scope or thread-local context remains available. Capture only the small immutable values the task needs.

Session scope caution

Session beans can increase memory, serialization, concurrency, and failover complexity. Multiple requests for one session may execute concurrently. Session scope does not serialize them. Keep authoritative business state in durable storage and session state minimal.

Scope versus ownership

Ask four questions:

1. Who creates the instance?
2. How many callers can access it concurrently?
3. Who releases resources?
4. Can a longer-lived object retain it past its intended boundary?

A request-scoped object leaked into a static field defeats the scope. A prototype owning a thread without explicit shutdown leaks resources even though instances are short-lived.

WATCH OUT | Common mistakes

- Saying Spring singleton is one instance for the whole JVM.
- Keeping mutable request state in singleton fields.
- Expecting prototype destruction callbacks.
- Injecting prototype directly into a singleton and expecting one per call.
- Passing request-scoped proxies into background threads.
- Using session scope as a database.

Interview angle

Interviewer: Are singleton beans thread-safe?

Strong answer: Scope and thread safety are separate. A Spring singleton is shared per bean definition per container, so concurrent callers can execute it. Stateless services are usually safe; shared mutable fields require an explicit concurrency design. I also inspect collaborators and the scope of any proxy rather than declaring every singleton safe.

Quick check

1. How does Spring singleton differ from GoF singleton?
2. When is a prototype created?
3. Who destroys a prototype?
4. Why is a scoped proxy useful?
5. Does session scope serialize requests?

Predict and debug

Predict: A singleton receives a prototype in its constructor. How many prototype instances does that singleton normally retain? One.

Debug: Tenant ID stored in a singleton field intermittently crosses requests. Remove request state from the shared bean; pass tenant context as an immutable argument or use a correctly bounded request context with strict async handling.

Practice

- **Foundation:** Prove two singleton lookups return one object and two prototype lookups return two.
- **Foundation:** Identify mutable fields in three service classes.
- **Interview Core:** Use `ObjectProvider` for one fresh operation object.
- **Interview Core:** Explain request proxy target resolution on two requests.
- **SDE-2 Follow-up:** Diagnose a cross-tenant leak involving singleton state and an executor.

TRY IT | Readiness checkpoint

Continue when you can separate scope, thread safety, and resource ownership and can predict prototype behavior inside a singleton.

SDE-2 CORE CHAPTER

Chapter 8 - Configuration Classes, Factory Methods, and Modularity

Java configuration is executable code, but Spring may enhance a full `@Configuration` class so inter-bean method calls preserve container semantics. This subtle mechanism appears often in senior interviews.

Full configuration mode

JAVA

```java
@Configuration
class ApplicationConfiguration {
    @Bean
    Repository repository() {
        return new JdbcRepository();
    }

    @Bean
    OrderService orderService() {
        return new OrderService(repository());
    }
}
```

In the default full mode, Spring creates a subclass-based enhancement of the configuration class. The call to `repository()` is intercepted and returns the container-managed singleton rather than constructing an extra repository.

That behavior is not normal Java. If you manually instantiate the configuration class, normal method calls apply and new objects can be created.

Parameter injection is clearer

JAVA

```java
@Bean
OrderService orderService(Repository repository) {
    return new OrderService(repository);
}
```

This expresses the dependency directly, works in full or lite configuration, and avoids relying on inter-bean method interception. Prefer it.

Lite configuration mode

An `@Bean` method declared in a non-`@Configuration` component, or an `@Configuration(proxyBeanMethods = false)` class, uses lite semantics. Direct Java calls between factory methods are not intercepted.

JAVA

```java
@Configuration(proxyBeanMethods = false)
class FastConfiguration {
    @Bean
    Repository repository() {
        return new JdbcRepository();
    }

    @Bean
    OrderService orderService(Repository repository) {
        return new OrderService(repository);
    }
}
```

This is safe because dependencies arrive as parameters. Lite mode can reduce configuration enhancement and restrictions, but correctness comes first.

Factory method parameters are injection points

Factory methods can receive beans, configuration values, and infrastructure types:

JAVA

```java
@Bean
PaymentClient paymentClient(
        HttpTransport transport,
        PaymentClientSettings settings) {
    return new PaymentClient(transport, settings.baseUri(), settings.timeout());
}
```

Keep factory methods deterministic and quick. They should not perform unbounded retries, migrations, or production data reads.

Configuration class constraints

Full configuration relies on subclass enhancement. Do not make full configuration classes or relevant methods final. Prefer a simple, package-visible configuration class with focused imports. If you need final classes, use `proxyBeanMethods = false` and parameter injection.

Modular assembly

TEXT

```
RootConfiguration
  |
  +-- OrdersConfiguration
  |     +-- domain services
  |     +-- repository port adapter
  |
  +-- PaymentsConfiguration
  |     +-- client settings
  |     +-- payment adapter
  |
  +-- RuntimeConfiguration
        +-- clock / executor / transaction manager
```

Each module should expose a small public configuration surface. Keep package internals unscanned from unrelated modules. Avoid a configuration class per individual bean; modularity should reflect a coherent capability.

Conditional registration boundary

Plain Framework offers profiles and programmable conditions. Spring Boot adds a larger conditional auto-configuration model. In core application configuration, prefer deterministic choices based on validated settings. If you implement a custom `Condition`, keep it free of side effects and produce diagnostic evidence about why it matched.

Bean method return types

Declare the narrow service interface when consumers should not depend on an implementation. For infrastructure extension points, declare a sufficiently precise type so Spring can detect their role without instantiating them early.

JAVA

```java
@Bean
static BeanFactoryPostProcessor propertyGuard() { ... }
```

WATCH OUT | Common mistakes

- Assuming every call to an `@Bean` method is container-intercepted.
- Using `proxyBeanMethods = false` while calling another factory method directly.
- Manually constructing a full configuration class in application code.
- Marking enhanced configuration final.
- Doing remote I/O in factory methods without bounded startup semantics.
- Splitting configuration so finely that no module boundary is visible.

Interview angle

Interviewer: Why might calling one `@Bean` method from another return the singleton?

Strong answer: In full `@Configuration` mode, Spring enhances the configuration class with a subclass that intercepts factory-method calls and routes them through the container. Lite mode does not. I normally use factory method parameters, which state dependencies directly and avoid relying on that interception.

Quick check

1. What makes full configuration behavior different from normal Java?
2. What does `proxyBeanMethods = false` change?
3. Why are factory method parameters clearer?
4. Which configuration mode supports final configuration classes more naturally?
5. What makes a useful configuration module?

Predict and debug

Predict: In lite mode, `return new OrderService(repository())` invokes ordinary Java and can create a repository not owned as the registered singleton.

Debug: Two connection pools exist despite one pool bean definition. Search for direct factory-method calls, manual `new Configuration()`, and adapters constructed outside the context; switch to parameter injection.

Practice

- **Foundation:** Rewrite an inter-bean call as a method parameter.
- **Foundation:** Explain full and lite mode in one diagram.
- **Interview Core:** Split a configuration into orders, payments, and runtime modules.
- **Interview Core:** Write a test proving one repository instance reaches two services.
- **SDE-2 Follow-up:** Diagnose duplicate expensive clients after a `proxyBeanMethods = false` optimization.

TRY IT | Readiness checkpoint

Continue when you can predict an `@Bean` call in full configuration, lite configuration, and a manually constructed configuration object.

SDE-2 CORE CHAPTER

Chapter 9 - Application Events and Decoupled Workflows

An application event communicates that something happened inside one application context. It can decouple the publisher from optional reactions, but it is not automatically durable, asynchronous, remote, or exactly once.

Publish a plain object event

JAVA

```java
record OrderPlaced(long orderId, String requestKey) { }

@Service
final class OrderService {
    private final ApplicationEventPublisher events;

    OrderService(ApplicationEventPublisher events) {
        this.events = events;
    }

    void place(long orderId, String requestKey) {
        // validate and persist the order
        events.publishEvent(new OrderPlaced(orderId, requestKey));
    }
}
```

Modern Spring events do not need to extend `ApplicationEvent`.

JAVA

```java
@Component
final class OrderMetricsListener {
    @EventListener
    void on(OrderPlaced event) {
        // record a bounded in-process metric
    }
}
```

Default execution model

With the default multicaster, listeners normally execute synchronously in the publisher's thread. A slow listener therefore slows the publisher; an unchecked listener failure can propagate back. The publisher API is a hand-off contract, so a customized multicaster may change execution. Know your configuration.

TEXT

```
publisher method
    |
    +-- publishEvent
            |
            +-- listener A (same thread by default)
            +-- listener B (same thread by default)
    |
    v
publisher continues
```

Events are not commands

A command asks one owner to perform required work. An event reports a fact that occurred. If payment authorization is required for order creation, hiding it in an optional event listener makes the business contract unclear. Use a direct dependency or explicit workflow.

Events fit reactions such as local cache invalidation, metrics, or separately owned follow-up behavior when failure semantics are deliberate.

Transaction-bound listeners

JAVA

```java
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
void afterOrderCommit(OrderPlaced event) {
    // react only if the publishing transaction committed
}
```

Phases include before commit, after commit, after rollback, and after completion. Without an active transaction, a transactional listener does not run by default unless fallback execution is enabled.

After-commit does not make a remote action durable. The database commit may succeed and the process may crash before an email or broker publish. For reliable cross-system delivery, write an outbox record in the same database transaction and let a separate relay publish it idempotently.

TEXT

```
transaction: order row + outbox row -> COMMIT
                                        |
                                        v
                                relay reads outbox
                                        |
                                        v
                                broker / email API
```

Ordering and idempotency

Listeners can be ordered, but a large chain of ordered listeners is an implicit workflow. Prefer an explicit orchestrator when later steps depend on earlier results. Make side-effecting listeners idempotent because retries or duplicate external delivery may occur around the process boundary.

Event design

- Use immutable payloads with stable identifiers.
- Do not publish mutable managed entities for later use.
- Do not put secrets or entire object graphs into events.
- Name past-tense facts: `OrderPlaced`, not `PlaceOrder`.
- Version durable external messages separately from in-process events.

WATCH OUT | Common mistakes

- Claiming `publishEvent` is asynchronous.
- Treating in-process events as a message broker.
- Performing mandatory business work in an optional listener.
- Assuming `AFTER_COMMIT` guarantees external delivery.
- Publishing a lazy entity and accessing it after its persistence context closes.
- Creating invisible workflow ordering through many listeners.

Interview angle

Interviewer: Would you send an email in an `@TransactionalEventListener(AFTER_COMMIT)`?

Strong answer: It avoids sending before a rolled-back order, but it still has a crash window after commit and before the email call. For best-effort notification it may be acceptable with telemetry. For reliable delivery, I commit an outbox record with the order, relay it asynchronously, and make the consumer idempotent. I also keep network I/O outside the database transaction.

Quick check

1. Are application events synchronous by default?
2. What is the command-versus-event distinction?
3. When does a transactional listener run without a transaction?
4. Why is after-commit not durable messaging?
5. What belongs in a safe event payload?

Predict and debug

Predict: A default synchronous listener throws an unchecked exception. It can fail the publishing call.

Debug: Orders commit but some emails vanish during deployments. Replace the process-memory hand-off with an outbox plus monitored idempotent relay.

Practice

- **Foundation:** Publish and listen for one immutable event.
- **Foundation:** Prove the default listener and publisher use the same thread.
- **Interview Core:** Classify three interactions as command, local event, or durable message.
- **Interview Core:** Test a listener that should run only after commit.
- **SDE-2 Follow-up:** Design outbox schema, relay lease, idempotency key, retries, and dead-letter handling.

TRY IT | Readiness checkpoint

Continue when you can state execution, transaction, durability, ordering, and retry semantics for every listener rather than assuming them from the annotation.

SDE-2 CORE CHAPTER

Chapter 10 - Conversion, Formatting, Validation, and Data Binding

External data begins as text, bytes, or loosely typed structures. A strong application converts it to typed values, validates structural constraints, then applies business invariants at a trusted boundary.

Conversion versus validation

TEXT

```
"2026-08-01" --convert--> LocalDate
       |                        |
 invalid syntax             validate rule
                          deliveryDate >= today
```

Conversion answers "can this representation become this type?" Validation answers "is this value acceptable for this use?" Business logic answers "is this state transition allowed now?"

ConversionService

Spring's type-conversion system supports standard conversions and custom converters.

JAVA

```java
record OrderId(long value) {
    OrderId {
        if (value <= 0) {
            throw new IllegalArgumentException("order ID must be positive");
        }
    }
}

final class StringToOrderId implements Converter<String, OrderId> {
    @Override
    public OrderId convert(String source) {
        return new OrderId(Long.parseLong(source));
    }
}
```

Register converters centrally so controllers and configuration share consistent rules. Conversion exceptions should become safe boundary errors rather than internal stack traces returned to callers.

Formatting is presentation-aware

Formatters convert text with locale context, commonly for numbers and dates. Keep canonical storage types independent of locale. `1,234.50` is not universally parsed the same way.

Spring Validator

JAVA

```java
record CreateOrderRequest(String requestKey, int quantity) { }

final class CreateOrderValidator implements Validator {
    @Override
    public boolean supports(Class<?> type) {
        return CreateOrderRequest.class.isAssignableFrom(type);
    }

    @Override
    public void validate(Object target, Errors errors) {
        CreateOrderRequest request = (CreateOrderRequest) target;
        ValidationUtils.rejectIfEmptyOrWhitespace(
                errors, "requestKey", "requestKey.required");
        if (request.quantity() <= 0) {
            errors.rejectValue("quantity", "quantity.positive");
        }
    }
}
```

Jakarta Bean Validation annotations such as `@NotBlank` and `@Positive` are also common when a provider is present. Spring integrates with that standard. Do not assume the annotation alone performs validation; a validation boundary must invoke it.

Structural versus business validation

Concern	Good location
required text, length, numeric range	request/command validation
ID syntax	conversion/binding boundary
order may transition from CREATED to PAID	domain/service invariant
request key unique under concurrency	database constraint plus service handling
caller may edit this tenant's order	authorization boundary

An existence check followed by insert is not a concurrency-safe uniqueness guarantee. Keep the database constraint.

Data binding risks

Data binding maps external property names to object properties. Binding directly to entities or broad mutable objects can allow **over-posting**: a client submits fields such as `role`, `tenantId`, `price`, or `status` that were never intended to be editable.

Use narrow request records/DTOs and explicit mapping:

JAVA

```java
record UpdateAddressRequest(String line1, String city, String postalCode) { }

void updateAddress(long customerId, UpdateAddressRequest request) {
    Customer customer = repository.require(customerId);
    customer.changeAddress(request.line1(), request.city(), request.postalCode());
}
```

The entity controls allowed mutation. Authorization and persistence remain explicit.

Error response design

Return stable error codes and field paths, not implementation details:

JSON

```json
{
  "code": "VALIDATION_FAILED",
  "errors": [
    {"field": "quantity", "code": "quantity.positive"}
  ]
}
```

Do not echo secret values, raw SQL, class names, or rejected payloads without redaction.

SpEL boundary

The Spring Expression Language can navigate properties, call methods, and evaluate expressions in configuration and framework features. It is powerful enough to become a security and maintainability problem. Never evaluate untrusted user input as SpEL. Prefer ordinary Java for business rules and use small, reviewed expressions only where the hosting feature requires them.

WATCH OUT | Common mistakes

- Treating conversion failure as business validation.
- Assuming Jakarta validation annotations execute automatically everywhere.
- Validating uniqueness only with a pre-query.
- Binding HTTP data directly to persistence entities.
- Returning rejected secret values in error details.
- Accepting user-authored SpEL.

Interview angle

Interviewer: Where do you validate an order request?

Strong answer: I convert syntax at the boundary, validate structural fields on a narrow request type, authorize the caller, enforce domain state transitions in the domain/service, and preserve concurrency-sensitive invariants with database constraints or atomic operations. I return stable safe errors and never bind arbitrary client fields onto an entity.

Quick check

1. How do conversion and validation differ?
2. What makes formatting locale-sensitive?
3. Why is a validation annotation not sufficient by itself?
4. What is over-posting?
5. Why can a uniqueness pre-check race?

Predict and debug

Predict: A request passes `@NotBlank` but violates a unique key at commit. Structural validation cannot eliminate a concurrent database race.

Debug: A user changed `isAdmin` by adding a JSON field. Replace entity binding with an allowlisted request DTO, explicit mapping, authorization, and tests for rejected unknown/protected fields.

Practice

- **Foundation:** Convert a string to a validated `OrderId`.
- **Foundation:** Validate required request key and positive quantity.
- **Interview Core:** Separate five rules into conversion, structural, domain, authorization, or database enforcement.
- **Interview Core:** Replace entity binding with an update request record.
- **SDE-2 Follow-up:** Design localized safe validation errors with stable machine codes and telemetry.

TRY IT | Readiness checkpoint

Continue when you can trace untrusted text through conversion, structural validation, authorization, domain invariants, and durable constraints without merging those responsibilities.

SDE-2 CORE CHAPTER

Chapter 11 - AOP: Join Points, Pointcuts, and Advice

Aspect-oriented programming applies one policy across many method calls without copying policy code into every service. Spring AOP is primarily **proxy-based method interception** for Spring-managed beans.

Vocabulary through one call

TEXT

```
caller -> proxy -> target method
           |
           +-- before advice
           +-- around advice (can proceed, replace, time, fail)
           +-- after returning / after throwing / finally-style advice
```

- **Join point:** a point where advice can run. In Spring AOP, think method execution reached through a proxy.
- **Pointcut:** a predicate selecting join points.
- **Advice:** behavior that runs at selected join points.
- **Aspect:** the pointcut and associated advice as one concern.
- **Target:** the underlying application object.
- **Proxy:** the object callers invoke so advice can run before delegating.

A timing aspect

JAVA

```java
@Aspect
@Component
final class TimingAspect {
    @Around("execution(* com.example.orders..*(..))")
    Object time(ProceedingJoinPoint call) throws Throwable {
        long start = System.nanoTime();
        try {
            return call.proceed();
        } finally {
            long elapsed = System.nanoTime() - start;
            System.out.println(call.getSignature() + " " + elapsed + "ns");
        }
    }
}
```

`call.proceed()` invokes the next interceptor or target. Forgetting it replaces the call. Calling it twice executes the target twice. Around advice therefore requires careful review.

Enable AspectJ annotation interpretation for Spring AOP:

JAVA

```java
@Configuration
@EnableAspectJAutoProxy
class AopConfiguration { }
```

The AspectJ annotation style here configures Spring proxy AOP; it does not automatically mean compile-time or load-time AspectJ weaving.

Pointcut design

Broad expression:

JAVA

```java
execution(* com.example..*(..))
```

This may advise configuration, infrastructure, getters, or framework callbacks unintentionally. Prefer an architectural boundary or marker annotation:

JAVA

```java
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@interface AuditedOperation { }

@Around("@annotation(AuditedOperation)")
Object audit(ProceedingJoinPoint call) throws Throwable { ... }
```

Marker annotations make opt-in visible but also couple the method to that policy metadata. Package or interface pointcuts keep application code unaware but can be too broad. Choose deliberately and test matching.

Advice ordering

Transactions, retries, authorization, metrics, and auditing may all wrap one call:

TEXT

```text
authorization -> retry -> transaction -> metrics -> target
```

This order is semantic. Retrying outside the transaction can create a fresh transaction per attempt; retrying inside may repeat work in one already-failed transaction, which is often wrong. State and test the intended nesting with `@Order` or ordered advisors. Do not rely on incidental registration order.

Good and poor AOP candidates

Good candidates are policies with a stable call boundary:

- transaction demarcation;
- authorization checks;
- metrics and tracing;
- bounded retries for classified failures;
- standardized auditing metadata.

Poor candidates hide domain workflow or change return values unexpectedly. If an aspect needs detailed knowledge of every method's business state, the concern may belong in explicit application code.

Exception and argument handling

Log or audit only safe metadata. A generic aspect sees arguments from many domains, including tokens, passwords, and personal data. Avoid serializing all arguments and return values. Preserve the original throwable unless the boundary has an explicit translation contract.

Testing an aspect

1. Unit-test pure policy helpers without Spring.
2. Start a focused context containing the target, aspect, and enablement.
3. Assert the bean is proxied.
4. Call through the context-provided reference.
5. Assert match and non-match cases, advice order, return value, and exception preservation.

WATCH OUT | Common mistakes

- Equating AspectJ annotations with AspectJ weaving.
- Using a pointcut so broad that infrastructure beans are advised.
- Forgetting or duplicating `proceed()`.
- Logging all arguments and leaking secrets.
- Assuming advice order.
- Using AOP to hide core business steps.

Interview angle

Interviewer: When would you use Spring AOP?

Strong answer: For a cross-cutting policy with a clear method boundary, such as transactions or metrics, when proxy interception is sufficient. I define a narrow, testable pointcut, document advice order, preserve failure semantics, and account for proxy limits such as self-invocation and non-advisable methods. I keep domain workflow explicit.

Quick check

1. What does a pointcut select?
2. What is the consequence of omitting `proceed()`?
3. Why is advice order part of correctness?
4. Does `@Aspect` imply bytecode weaving?
5. Which data should a generic logging aspect avoid?

Predict and debug

Predict: Around advice calls `proceed()` twice. The target can execute twice and duplicate side effects.

Debug: A metric appears on every configuration method. Narrow the package, interface, or annotation pointcut and add a non-match test.

Practice

- **Foundation:** Label caller, proxy, advice, and target in a diagram.
- **Foundation:** Write around advice that always calls `proceed()` exactly once.
- **Interview Core:** Design match and non-match tests for an audit annotation.
- **Interview Core:** Choose ordering for authorization, retry, transaction, and metrics.
- **SDE-2 Follow-up:** Replace an aspect that logs payloads with a privacy-safe structured telemetry contract.

TRY IT | Readiness checkpoint

Continue when you can explain what call the aspect intercepts, what it may change, how advisors are ordered, and which tests prove the boundary.

SDE-2 CORE CHAPTER

Chapter 12 - Proxy Mechanics, Self-Invocation, and Runtime Types

Most surprising Spring behavior is ordinary Java dispatch around an extra object. Draw the proxy before changing annotations.

JDK and subclass proxies

Spring AOP can use:

- **JDK dynamic proxy:** implements interfaces exposed by the target.
- **Subclass proxy:** runtime-generated subclass of the target class, using repackaged CGLIB support.

TEXT

```
JDK proxy                          subclass proxy
implements OrderOperations       extends OrderService
          |                                |
          v                                v
       target                         target logic
```

With interfaces, core Spring AOP commonly chooses JDK proxies by default. Class-based proxying can be requested. Spring Boot may apply its own defaults; do not project Boot behavior onto every Framework context.

The external-call requirement

JAVA

```java
@Service
class BillingService {
    void billAll() {
        billOne();             // this.billOne(): target-to-target call
    }

    @AuditedOperation
    public void billOne() { }
}
```

When an external caller invokes `proxy.billOne()`, advice can run. After control reaches the target, `billAll()` calling `this.billOne()` does not re-enter the proxy. The advice on `billOne` is bypassed.

TEXT

```
external caller -> proxy -> target.billAll
                                |
                                +-> this.billOne  (no proxy crossing)
```

Preferred correction: extract the advised operation into a separate collaborator and call it through its injected proxy.

JAVA

```java
@Service
final class SingleBillingService {
    @AuditedOperation
    public void billOne() { }
}

@Service
final class BatchBillingService {
    private final SingleBillingService single;

    BatchBillingService(SingleBillingService single) {
        this.single = single;
    }

    void billAll() {
        single.billOne();
    }
}
```

Self-injection and `AopContext.currentProxy()` exist but couple design to the proxy mechanism and are weaker defaults.

Methods that cannot be advised reliably

For class-based proxies:

- final classes cannot be subclassed;
- final methods cannot be overridden;
- private methods cannot be overridden;
- methods not visible to the subclass may not be advised.

JDK proxies expose interface methods. A caller typed to a concrete class may fail or bypass the intended contract when the bean is a JDK proxy. Design application ports as interfaces when that abstraction is real; do not create meaningless interfaces solely to satisfy a myth that Spring requires them.

Runtime class surprises

```java
Object bean = context.getBean("orderService");
System.out.println(bean.getClass());
```

The class may be a generated proxy, not exactly `OrderService.class`. Avoid exact-class equality for behavior decisions. Use interfaces, `instanceof` thoughtfully, and Spring utilities when infrastructure must resolve a target class. Domain code should not inspect proxy types.

Annotations and interface placement

For consistent transaction discovery across proxy styles, place annotations on concrete implementation methods/classes as recommended by Spring documentation, unless your framework contract and tests explicitly support interface annotations. Meta-annotations and inheritance rules can become subtle; verify the actual method Spring resolves.

Proxy chains and identity

One bean may have several advisors, usually in one proxy chain. Avoid manually wrapping Spring proxies in unrelated proxies without understanding equality, serialization, and type exposure. `equals` and `hashCode` should reflect domain/value semantics, not target/proxy identity assumptions.

Diagnosing an annotation that does nothing

Use this checklist:

1. Is the object a Spring bean or created with `new`?
2. Is the feature enabled and its post-processor registered?
3. Did the caller receive the proxy?
4. Is this an external call or self-invocation?
5. Is the method visible and overridable for the proxy type?
6. Does the pointcut/annotation match the resolved method?
7. Is a later post-processor replacing the bean unexpectedly?
8. Does a focused test assert advice execution rather than only successful business output?

WATCH OUT | Common mistakes

- Saying Spring modifies the target method body.
- Expecting private/final methods to be advised by subclass proxies.
- Calling an advised method from the same object.
- Casting a JDK proxy to its concrete implementation.
- Checking exact runtime class in business code.
- Assuming adding an annotation enables its infrastructure.

Interview angle

Interviewer: Why does `@Transactional` work from a controller but not from another method in the same service?

Strong answer: In the normal proxy mode, the controller calls the Spring proxy, so transaction advice intercepts the call. A same-instance call uses `this` after control reached the target and bypasses the proxy. I move the transactional operation to a separate bean or place the transaction at the externally invoked orchestration method, then prove it with a rollback test.

Quick check

1. What does a JDK proxy expose?
2. Why does self-invocation bypass advice?
3. Why can final methods be a problem for subclass proxies?
4. Should business code inspect generated proxy classes?
5. What is the first diagnostic question for an ineffective annotation?

Predict and debug

Predict: A JDK proxy implements `OrderOperations`; casting it to `OrderService` can throw `ClassCastException`.

Debug: An audit annotation on a private helper never runs. Put the policy boundary on an externally invoked public operation or make the behavior explicit; do not expose internals solely for proxying.

Practice

- **Foundation:** Draw external proxy call and self-invocation paths.
- **Foundation:** Inspect whether a context bean is an AOP proxy.
- **Interview Core:** Refactor one self-invoking service into two collaborators.
- **Interview Core:** Test interface-based and class-based proxy type exposure.
- **SDE-2 Follow-up:** Diagnose a production policy bypass introduced when a method became final.

TRY IT | Readiness checkpoint

Continue when you can predict interception from object ownership, call path, proxy type, and method visibility without guessing from the annotation.

SDE-2 CORE CHAPTER

Chapter 13 - Transaction Abstraction and Declarative Boundaries

A transaction protects a business invariant across a set of resource operations. `@Transactional` describes a boundary; a `PlatformTransactionManager` implements begin, participation, commit, and rollback for a specific resource strategy.

WHY IT WORKS | Start from the invariant

Placing an order writes an order and reserves inventory. The invariant is not "both repository methods have annotations." It is:

> Either the order and reservation both commit, or neither becomes durable.

TEXT

```
controller
    |
    v
transaction proxy
    |
    +-- BEGIN
    +-- target OrderApplicationService.place(...)
    |       +-- insert order
    |       +-- update inventory
    +-- COMMIT
    |
    +-- on qualifying failure: ROLLBACK
```

The application-service operation is usually the right boundary because it owns the full use case.

Minimal declarative setup

JAVA

```java
@Configuration
@EnableTransactionManagement
class TransactionConfiguration {
    @Bean
    PlatformTransactionManager transactionManager(DataSource dataSource) {
        return new DataSourceTransactionManager(dataSource);
    }
}
```

```java
@Service
final class OrderApplicationService {
    private final OrderRepository orders;
    private final InventoryRepository inventory;

    @Transactional
    public long place(PlaceOrder command) {
        long orderId = orders.insert(command);
        inventory.reserve(command.sku(), command.quantity());
        return orderId;
    }
}
```

For JPA, use the appropriate transaction manager that coordinates the `EntityManager`. Do not combine transaction managers casually and assume one annotation creates atomicity across them.

Runtime sequence

1. Caller invokes the Spring proxy.
2. Transaction interceptor resolves transaction attributes.
3. Manager opens or joins a transaction and binds resource state to the execution context, commonly the current thread for imperative transactions.
4. Target method executes.
5. Normal return leads to commit unless marked rollback-only.
6. A configured throwable leads to rollback.
7. Resources are cleaned up and unbound.

The database owns actual isolation, locks, constraints, and durability. Spring coordinates access; it does not redefine ACID.

Declarative and programmatic styles

Declarative transactions keep ordinary service code focused:

```java
@Transactional
public void transfer(...) { ... }
```

`TransactionTemplate` makes boundaries explicit when one method needs multiple differently scoped units:

```java
long id = transactionTemplate.execute(status -> orders.create(command));
remoteClient.notify(id); // outside database transaction
```

Programmatic transaction code is not inherently superior. Use it when boundary shape truly requires explicit control, not to work around misunderstood proxies.

Rollback defaults

By default, Spring declarative transactions roll back for unchecked `RuntimeException` and `Error`, not ordinary checked exceptions. Customize when a checked business/infrastructure exception must roll back:

JAVA

```java
@Transactional(rollbackFor = PaymentFileException.class)
public void importPaymentFile(...) throws PaymentFileException { ... }
```

Do not add `rollbackFor = Exception.class` everywhere without examining which exceptions mean failure versus an expected alternate result.

Catching exceptions changes the outcome

JAVA

```java
@Transactional
public void place(PlaceOrder command) {
    try {
        inventory.reserve(command.sku(), command.quantity());
    } catch (RuntimeException failure) {
        log.warn("reservation failed", failure);
        // method returns normally: transaction may commit prior work
    }
}
```

If the exception is swallowed, the interceptor sees a normal return unless inner infrastructure already marked the transaction rollback-only. Either translate and rethrow according to contract, or explicitly mark rollback-only only when that coupling is justified.

Transaction boundaries and remote calls

Holding a database transaction open during an HTTP call increases connection hold time, lock duration, latency variance, and failure coupling. Prefer:

TEXT

```
short DB transaction -> commit intent/outbox -> remote delivery -> idempotent status update
```

When a remote response must determine the database change, model a state machine and compensating/reconciliation behavior. Spring transactions cannot atomically commit an ordinary HTTP API.

WATCH OUT | Common mistakes

- Annotating private helpers and expecting proxy advice.
- Spreading transaction boundaries across repository methods so one use case is not atomic.
- Catching failures and accidentally committing partial work.
- Keeping transactions open during slow remote calls.
- Assuming one transaction manager covers multiple databases or a broker.
- Testing only successful writes, never rollback.

Interview angle

Interviewer: Where do you put `@Transactional`?

Strong answer: At an externally invoked application-service method that owns one business invariant and one intentionally bounded resource unit. Repositories participate in that boundary. I keep remote I/O outside, choose the correct manager, state rollback rules, and integration-test both durable commit and rollback rather than merely checking that the annotation exists.

Quick check

1. Who owns actual database locks and isolation?
2. What are default rollback throwable categories?
3. Why can catching an exception cause partial commit?
4. When is `TransactionTemplate` useful?
5. Can a JDBC transaction atomically include an HTTP call?

Predict and debug

Predict: A checked exception leaves the method under default rules. Unless configured otherwise or marked rollback-only, the transaction can commit.

Debug: Order row persists after inventory failure. Inspect call-through-proxy, manager choice, exception translation/catching, propagation, and an integration test against committed database state.

Practice

- **Foundation:** Mark begin, statements, commit, and rollback on a service sequence.
- **Foundation:** Write a rollback integration test for two writes.
- **Interview Core:** Refactor repository-level transactions into one use-case boundary.
- **Interview Core:** Move one remote call outside a database transaction.
- **SDE-2 Follow-up:** Design an order/outbox workflow with idempotency and reconciliation.

TRY IT | Readiness checkpoint

Continue when you can state invariant, proxy entry, manager, resource, rollback rule, and external side-effect boundary for one transaction.

SDE-2 CORE CHAPTER

Chapter 14 - Propagation, Isolation, Rollback, and Timeouts

Transaction attributes are not decoration. They change resource participation and failure behavior. Begin with `REQUIRED`, then use another mode only for a named reason.

Propagation mental model

Propagation	Existing transaction	No transaction	Primary use/caution
`REQUIRED`	join	create	normal application use case
`REQUIRES_NEW`	suspend, create independent	create	independent durable unit; needs another resource/connection
`SUPPORTS`	join	run without	operations valid either way; semantics vary by caller
`MANDATORY`	join	fail	require caller-owned boundary
`NOT_SUPPORTED`	suspend, run without	run without	explicitly non-transactional work
`NEVER`	fail	run without	enforce absence
`NESTED`	savepoint within physical transaction when supported	create	partial rollback through savepoint; manager/resource dependent

REQUIRED and rollback-only surprise

JAVA

```java
@Transactional
public void placeOrder(...) {
    paymentLedger.record(...); // REQUIRED
    // caller catches an inner failure and continues
}
```

Joined scopes share one physical transaction. If the inner scope marks it rollback-only and the outer scope still attempts commit, Spring throws `UnexpectedRollbackException`. This is deliberate: the caller must not be told that commit succeeded.

REQUIRES_NEW is independent, not free

JAVA

```java
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void writeAudit(...) { ... }
```

The outer resource is suspended while a new transaction obtains its own resources. Under concurrency, each thread may hold an outer connection while requesting another, exhausting a small pool. The inner commit remains even if the outer transaction later rolls back. That may be correct for audit evidence or incorrect for business state.

Also remember the proxy boundary: a same-class call to this method does not activate new propagation in proxy mode.

NESTED versus REQUIRES_NEW

`NESTED` typically uses a database savepoint inside one physical transaction. Rolling back to the savepoint does not commit the inner work independently; the outer final rollback still removes everything. Support depends on transaction manager and resource capabilities. `REQUIRES_NEW` uses an independent physical transaction.

Isolation belongs to the database contract

Spring exposes isolation choices such as read uncommitted, read committed, repeatable read, and serializable. Their exact phenomena, locking, snapshots, and defaults come from the database and access path. Review **MySQL Book 02** for InnoDB details.

Do not set `SERIALIZABLE` as a reflex. First name the anomaly or invariant, then compare:

- database constraint;
- atomic conditional update;
- optimistic version;
- explicit lock;
- higher isolation;
- redesigned workflow.

readOnly

`@Transactional(readOnly = true)` is a hint and optimization contract whose effect depends on the manager, provider, driver, and database. It is not a security boundary and must not be your only write prevention. Tests should verify critical behavior rather than assuming universal enforcement.

Timeout

A transaction timeout bounds transaction execution according to manager/resource support. It is not a complete end-to-end request deadline. Also configure database statement/lock timeouts, client timeouts, queue time, and cancellation semantics. A request that times out in a caller may still be executing unless cancellation propagates.

Rollback rules and exception translation

Use type-safe rollback rules where possible. Pattern-based class-name rules can match more broadly than intended. Translate low-level exceptions at a stable boundary while preserving category and cause:

JAVA

```java
try {
    repository.insert(order);
} catch (DuplicateKeyException duplicate) {
    throw new DuplicateRequestKey(command.requestKey(), duplicate);
}
```

The translated exception should still trigger the intended rollback.

Manager selection

With multiple `PlatformTransactionManager` beans, choose explicitly:

JAVA

```java
@Transactional(transactionManager = "ordersTransactionManager")
public void updateOrder(...) { ... }
```

This does not create atomicity with a second manager. Distributed transaction support is a separate architecture decision with significant operational cost; many services use local transactions plus outbox, idempotency, and compensation.

WATCH OUT | Common mistakes

- Using `REQUIRES_NEW` to make errors disappear.
- Confusing nested savepoint with independent commit.
- Ignoring connection-pool demand from suspended outer transactions.
- Treating `readOnly` as authorization.
- Assuming timeout covers remote calls and queueing end to end.
- Selecting propagation on a self-invoked method.

Interview angle

Interviewer: Explain `REQUIRED` versus `REQUIRES_NEW`.

Strong answer: `REQUIRED` joins the caller's transaction or creates one, so all participants share commit and rollback. `REQUIRES_NEW` suspends the outer transaction and commits independently with separate resource demand. I use it only when independent durability is a business requirement, account for pool capacity and lock interactions, and ensure the invocation crosses a proxy.

Quick check

1. Why can `UnexpectedRollbackException` be correct?
2. What resource risk does `REQUIRES_NEW` add?
3. How does `NESTED` differ from an independent transaction?
4. Is `readOnly` a write-security guarantee?
5. Why is isolation database-specific?

Predict and debug

Predict: Inner `REQUIRED` marks rollback-only; outer catches and returns. The final commit attempt produces `UnexpectedRollbackException` rather than a false success.

Debug: Audit traffic exhausts the pool after adding `REQUIRES_NEW`. Measure active/awaiting connections, outer hold time, concurrency, and pool/database capacity; reconsider whether independent audit writes belong in an outbox.

Practice

- **Foundation:** Trace `REQUIRED` with and without an existing transaction.
- **Foundation:** Contrast new transaction and savepoint on a timeline.
- **Interview Core:** Explain an unexpected rollback from inner failure.
- **Interview Core:** Choose an isolation/invariant strategy for last-item inventory.
- **SDE-2 Follow-up:** Capacity-plan outer plus `REQUIRES_NEW` connection demand and design a safer alternative.

TRY IT | Readiness checkpoint

Continue when you can trace physical transaction, logical scope, resource count, rollback-only state, and final durable outcome for nested service calls.

SDE-2 CORE CHAPTER

Chapter 15 - Transaction Failure, Retries, and Production Patterns

Production transaction design begins where the happy-path annotation ends: deadlocks, lock timeouts, duplicate requests, uncertain remote outcomes, and partial delivery.

A failed transaction is finished

After a database exception marks a transaction rollback-only, do not catch it and continue issuing work as if the transaction were healthy. Roll back, release state, classify the failure, and decide whether the **whole idempotent operation** can run in a fresh transaction.

```text
attempt 1: BEGIN -> read -> write -> deadlock -> ROLLBACK
                     wait with bounded jitter
attempt 2: BEGIN -> reread current state -> write -> COMMIT
```

Retrying one statement inside the same failed transaction is usually incorrect.

Classify before retrying

Failure	Retry automatically?	Required reasoning
deadlock victim	often, bounded	whole transaction idempotent; fresh state
transient lock timeout	sometimes	contention and latency budget
connection interruption	uncertain	commit outcome may be unknown
unique constraint	usually no	may represent idempotent replay or business conflict
validation/authorization	no	deterministic caller error
optimistic conflict	maybe	command still semantically valid after reload
serialization failure	often bounded	transaction replay is safe

The exception class alone is insufficient. Driver/database translation, SQL state, operation semantics, and whether commit acknowledgement was received all matter.

Retry and transaction advice ordering

The intended shape is usually:

TEXT

```
retry advice
   |
   +-- attempt 1 transaction -> rollback
   +-- wait
   +-- attempt 2 transaction -> commit
```

If retry runs inside one transaction, later attempts may reuse a rollback-only context. Make ordering explicit and test the number of begins/rollbacks/commits.

Idempotency key pattern

TEXT

```
request key: customer-42/place-order/abc123
        |
        v
UNIQUE constraint on request key
        |
        +-- first request inserts result
        +-- replay reads and returns the same result
        +-- same key + different payload -> conflict
```

Store a payload fingerprint and stable outcome where needed. A plain "duplicate means success" rule can return the wrong result when the same key is reused for different input.

Remote call uncertainty

Three outcomes exist after a payment timeout:

1. Provider never received the request.
2. Provider completed it but response was lost.
3. Provider is still processing.

Blind retry can charge twice. Use provider-supported idempotency keys, durable local attempt state, status lookup/reconciliation, and an explicit state machine.

TEXT

```
CREATED -> PAYMENT_PENDING -> PAID
                    |
                    +-> UNKNOWN -> reconcile -> PAID or FAILED
```

Outbox pattern

Write domain state and an outbox message in one local transaction. A relay claims unpublished rows, publishes with a stable message ID, and marks them sent. The relay can crash after publish but before marking sent, so consumers must be idempotent.

Key design points:

- immutable event payload and schema version;
- aggregate/order key when ordering matters;
- claim/lease that recovers after crash;
- bounded retries and poison-message policy;
- backlog age, attempt count, and publish latency metrics;
- retention and personally identifiable information controls.

Transaction observability

Measure:

- active, committed, rolled-back, timed-out transaction counts;
- duration and connection acquisition time;
- lock/deadlock/serialization failure category;
- retry attempts and exhausted retries;
- outbox backlog age and relay failures;
- operation name without high-cardinality raw IDs.

A stack trace without transaction name, manager, SQL state/category, attempt number, and resource timing is weak evidence.

Incident playbook: rollback spike

1. Contain overload or disable the offending workflow safely.
2. Separate acquisition, execution, lock wait, and commit latency.
3. Group failures by operation and translated exception category.
4. Inspect recent deployment, schema/index, traffic, and query-plan changes.
5. Determine whether retries amplify load.
6. Correct the invariant/query/boundary, canary, and watch rollback plus success latency.
7. Add a regression test and operational guardrail.

WATCH OUT | Common mistakes

- Retrying every `DataAccessException`.
- Retrying inside the same failed transaction.
- Retrying a non-idempotent remote side effect.
- Treating unique violation as universally transient or fatal.
- Assuming timeout means the remote operation failed.
- Marking outbox sent before publish succeeds.

Interview angle

Interviewer: How do you retry a deadlocked transaction?

Strong answer: I let the attempt roll back completely, classify the database exception, verify the command and external effects are idempotent, wait with bounded jitter, then rerun the entire unit in a fresh transaction against current state. I cap attempts and latency, expose metrics, and fix persistent contention or access-order issues instead of treating retries as the solution.

Quick check

1. Why must a retry use a fresh transaction?
2. What makes a command idempotent?
3. Why is an HTTP timeout an uncertain outcome?
4. Can an outbox deliver exactly once by itself?
5. Which metrics reveal retry amplification?

Predict and debug

Predict: Relay publishes and crashes before setting `sent_at`. It republishes later. The consumer needs a message ID and idempotent effect.

Debug: Deadlock retries raise database load and p99. Inspect retry rate, contention keys, lock order, query plans, transaction length, and synchronized retry bursts; cap attempts and fix the underlying conflict.

Practice

- **Foundation:** Classify five failures as deterministic, transient, conflict, or uncertain.
- **Foundation:** Draw a fresh-transaction retry timeline.
- **Interview Core:** Design an idempotency record with payload fingerprint.
- **Interview Core:** Model a payment `UNKNOWN` state and reconciliation.
- **SDE-2 Follow-up:** Write an outbox incident runbook with recovery and duplicate handling.

TRY IT | Readiness checkpoint

Continue when you can explain not only rollback but also replay safety, uncertainty, durable hand-off, resource demand, and measurable recovery.

SDE-2 CORE CHAPTER

Chapter 16 - Spring MVC Request Flow and HTTP Boundaries

Spring MVC is part of Spring Framework. Spring Boot can configure it conveniently, but the request pipeline belongs to Framework and should not be treated as hidden magic.

Request flow

TEXT

```
HTTP client
    |
    v
Servlet container filters
    |
    v
DispatcherServlet
    |
    +-> HandlerMapping finds controller method
    +-> HandlerAdapter resolves arguments and invokes it
    |       +-> conversion / binding / validation
    |       +-> application service
    +-> return-value handling / message conversion
    +-> exception resolvers when needed
    |
    v
HTTP response
```

`DispatcherServlet` is the front controller. It delegates to strategies; it does not contain every application behavior itself.

A small controller

JAVA

```java
@RestController
@RequestMapping("/orders")
final class OrderController {
    private final OrderApplicationService orders;

    OrderController(OrderApplicationService orders) {
        this.orders = orders;
    }

    @PostMapping
    ResponseEntity<OrderResponse> create(
            @Valid @RequestBody CreateOrderRequest request) {
        OrderResponse created = orders.create(request);
        URI location = URI.create("/orders/" + created.id());
        return ResponseEntity.created(location).body(created);
    }

    @GetMapping("/{id}")
    OrderResponse find(@PathVariable long id) {
        return orders.require(id);
    }
}
```

The controller translates HTTP into an application command/query and translates the result back. It should not own SQL, multi-step domain rules, or remote retry loops.

Parameter sources are different contracts

- `@PathVariable`: identifies a resource in the route.
- `@RequestParam`: query/form parameter, often filtering or optional control.
- `@RequestHeader`: protocol metadata; avoid making core domain state header-only.
- `@RequestBody`: content decoded by an HTTP message converter.
- Framework-provided types: request context, locale, authentication principal when configured.

Never assume a missing value becomes a safe Java default. Define required/optional behavior and validate ranges.

DTOs protect the boundary

JAVA

```java
record CreateOrderRequest(
        @NotBlank String requestKey,
        @NotBlank String sku,
        @Positive int quantity) { }
```

Do not expose persistence entities as request/response models. DTOs avoid lazy-loading during serialization, over-posting, accidental internal fields, and schema coupling.

HTTP semantics

Use status codes to express the contract:

Situation	Typical response
created resource	`201 Created` plus `Location`
successful read	`200 OK`
no response body	`204 No Content`
malformed/invalid input	`400 Bad Request`
unauthenticated	`401 Unauthorized`
authenticated but forbidden	`403 Forbidden`
missing resource	`404 Not Found`
state/version/idempotency conflict	`409 Conflict`
unsupported media type	`415 Unsupported Media Type`
unexpected server failure	`500 Internal Server Error`

The exact API contract can refine these choices. Do not return `200` with an error string for every outcome.

Central exception translation

JAVA

```java
@RestControllerAdvice
final class ApiExceptionHandler {
    @ExceptionHandler(OrderNotFound.class)
    ResponseEntity<ProblemDetail> notFound(OrderNotFound failure) {
        ProblemDetail problem = ProblemDetail.forStatus(404);
        problem.setTitle("Order not found");
        problem.setProperty("code", "ORDER_NOT_FOUND");
        return ResponseEntity.status(404).body(problem);
    }
}
```

Map known application failures precisely. Log unexpected failures once at the owning boundary with a correlation/trace identifier, then return a safe generic problem. Never return stack traces, SQL, secrets, or internal class names.

Filters, interceptors, and controller advice

Mechanism	Boundary	Good use
Servlet filter	before/after Spring MVC, raw request/response	correlation ID, low-level protocol/security filter chain
`HandlerInterceptor`	mapped MVC handler execution	handler-aware timing or policy metadata
controller advice	controller binding/return/exception concerns	consistent API exception and binding responses
AOP service advice	Spring bean method	transaction, service policy, metrics

Authentication and authorization should use Spring Security rather than custom filter inventions; full security is in **SD 10 - Spring Ecosystem Extensions**.

Idempotent create endpoints

For retryable clients, accept an idempotency key, bind it to caller plus operation, store a payload fingerprint and result under a uniqueness constraint, and return the prior result for a true replay. Do not hold an HTTP request open inside a long database transaction unnecessarily.

WATCH OUT | Common mistakes

- Returning entities directly.
- Putting business orchestration in controllers.
- Using `@ControllerAdvice` to swallow every exception as `200`.
- Treating filters, interceptors, and aspects as interchangeable.
- Logging full bodies or authorization headers.
- Assuming request validation prevents database conflicts.

Interview angle

Interviewer: Walk through a Spring MVC request.

Strong answer: The servlet container runs filters and dispatches to `DispatcherServlet`. Handler mappings choose a controller method, a handler adapter resolves and converts arguments, validation runs when configured, the controller invokes an application boundary, return-value handlers and message converters produce the response, and exception resolvers translate failures. I keep DTO, status, auth, transaction, and error contracts explicit.

Quick check

1. What role does `DispatcherServlet` play?
2. Why are entities poor API DTOs?
3. Which layer should own a database transaction?
4. How do a filter and interceptor differ?
5. What must a safe error response exclude?

Predict and debug

Predict: `@Valid` is omitted from a body parameter. Bean Validation constraints on the DTO may not be invoked for that parameter.

Debug: Serializing an order triggers 100 queries. Map a bounded DTO inside the service transaction and fetch/project exactly the response shape; do not make all associations eager.

Practice

- **Foundation:** Map create and find routes with appropriate statuses.
- **Foundation:** Replace an entity response with a record DTO.
- **Interview Core:** Translate validation, not-found, conflict, and unexpected errors.
- **Interview Core:** Place correlation, authorization, transaction, and exception logic at correct boundaries.
- **SDE-2 Follow-up:** Design an idempotent create endpoint with concurrent duplicate requests.

TRY IT | Readiness checkpoint

Continue when you can trace one request from filter to response and explain why each concern belongs at its chosen boundary.

SDE-2 CORE CHAPTER

Chapter 17 - Async Execution, Scheduling, and Context Boundaries

Asynchronous execution moves work to an executor. Scheduling decides when work becomes eligible to run. Neither feature supplies durability, capacity, ordering, or context propagation automatically.

Enable only what you use

JAVA

```java
@Configuration
@EnableAsync
@EnableScheduling
class ExecutionConfiguration {
    @Bean("emailExecutor")
    Executor emailExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(4);
        executor.setMaxPoolSize(8);
        executor.setQueueCapacity(200);
        executor.setThreadNamePrefix("email-");
        executor.initialize();
        return executor;
    }
}
```

The numbers are examples, not recommendations. Size concurrency from downstream capacity, service time, memory per task, and latency target. An unbounded queue converts overload into growing latency and memory pressure.

@Async call flow

JAVA

```java
@Service
final class EmailService {
    @Async("emailExecutor")
    public CompletableFuture<DeliveryResult> send(EmailCommand command) {
        return CompletableFuture.completedFuture(deliver(command));
    }
}
```

TEXT

```
caller -> async proxy -> submit task -> caller receives Future
                         |
                         v
                  executor thread -> target method
```

The default annotation mode is proxy-based, so self-invocation bypasses `@Async`. The returned future represents task completion, not durable acceptance. Process crash can lose queued memory tasks.

Failure semantics

For `Future`/`CompletableFuture`, failures complete the future exceptionally and must be observed. For `void` async methods, exceptions cannot return to the caller; configure an `AsyncUncaughtExceptionHandler` and telemetry. Prefer a result-bearing type or durable queue for important work.

Thread-local and transaction boundaries

Imperative transaction state and many request/security/logging contexts are thread-bound. Moving to an executor does not copy them automatically.

TEXT

```
request thread: transaction + request context
        |
        +-- submit immutable command
                 |
                 v
executor thread: new context; start its own transaction if needed
```

Capture stable IDs and immutable data, not an open `EntityManager`, request-scoped bean, mutable entity, or thread-local assumption. If context propagation is needed, use a deliberate supported mechanism and test cleanup to prevent one task's data from leaking into another.

@Scheduled

JAVA

```java
@Scheduled(fixedDelay = 30, timeUnit = TimeUnit.SECONDS)
public void relayOutbox() {
    // claim a bounded batch and process idempotently
}
```

- **fixed delay:** next run is measured after prior completion.
- **fixed rate:** attempts to maintain a rate measured from start times; overlap/backlog depends on scheduler configuration.
- **cron:** calendar schedule; specify timezone when business meaning depends on it.

Scheduled methods are commonly no-argument operations. A schedule is not a cluster-wide singleton. Ten service replicas can each run it unless coordination, sharding, or leader/lease ownership is designed.

Durable scheduled work

A reliable job needs:

- bounded claim size and stable ordering;
- database lease/claim with expiration or an external scheduler;
- idempotent processing;
- checkpoint and restart behavior;
- poison-item policy;
- concurrency limit and backpressure;
- lag, duration, outcome, and oldest-item metrics.

Do not keep one transaction around an entire million-row job. Claim/process in bounded units and define whether each item or batch is atomic.

Cancellation and shutdown

Interruption is cooperative. Code must honor cancellation when safe. Configure graceful executor shutdown and a bounded wait, but also design queued work to survive or be safely retried after process termination if it matters.

WATCH OUT | Common mistakes

- Assuming `@Async` means durable messaging.
- Using an unbounded executor queue.
- Ignoring exceptional future completion.
- Sharing a transaction or persistence context across threads.
- Assuming one scheduled invocation across a cluster.
- Scheduling long work without leases, idempotency, or backlog metrics.

Interview angle

Interviewer: What breaks when you add `@Async` to a transactional method call?

Strong answer: The call must first cross the async proxy. Work then runs on another thread, so the caller's imperative transaction and thread-local request context do not propagate automatically. The async method can start its own transaction, but that transaction is independent. I pass an immutable command/ID, define failure observation and executor capacity, and use a durable queue if losing in-memory work is unacceptable.

Quick check

1. Why can self-invocation bypass `@Async`?
2. How are async exceptions observed?
3. Does transaction state follow an executor task?
4. What is fixed delay measured from?
5. Why is `@Scheduled` not cluster coordination?

Predict and debug

Predict: A scheduled method runs in each of six replicas. Without coordination, the logical job executes six times.

Debug: Async email backlog grows while CPU is idle. Measure queue depth/age, downstream latency/rate limits, active threads, rejection, and connection pools; increasing threads may only overload the provider.

Practice

- **Foundation:** Configure a named bounded executor and return a `CompletableFuture`.
- **Foundation:** Contrast fixed delay, fixed rate, and cron.
- **Interview Core:** Refactor an async method to accept an immutable ID rather than an entity.
- **Interview Core:** Design exceptional completion and safe telemetry.
- **SDE-2 Follow-up:** Build a multi-replica outbox scheduler using leases, idempotency, and backpressure.

TRY IT | Readiness checkpoint

Continue when you can state executor, queue, thread, transaction, context, durability, cancellation, and overload behavior for each async or scheduled task.

SDE-2 CORE CHAPTER

Chapter 18 - Testing: From Pure Units to Spring Contexts

A Spring test is valuable only when Spring behavior is the subject. Most business logic should be tested as ordinary Java; container, proxy, transaction, and MVC contracts need focused integration tests.

The test ladder

TEXT

```
many: pure Java unit tests
       |
       v
focused context tests for wiring/proxies/conversion
       |
       v
resource integration tests for database/HTTP/message behavior
       |
       v
few: end-to-end tests across deployed boundaries
```

More infrastructure means slower feedback and more failure causes. Use the lowest level that can disprove the risk.

Pure unit test

JAVA

```java
@Test
void rejectsNonPositiveQuantity() {
    OrderRepository repository = new InMemoryOrderRepository();
    OrderService service = new OrderService(repository, Clock.fixed(
            Instant.parse("2026-07-29T12:00:00Z"), ZoneOffset.UTC));

    assertThrows(IllegalArgumentException.class,
            () -> service.place("book", 0));
}
```

No context, annotations, or mocks are required if a small fake expresses the contract.

Focused context test

JAVA

```java
@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = TestConfiguration.class)
final class WiringTest {
    @Autowired
    OrderService service;

    @Test
    void contextWiresService() {
        assertNotNull(service);
    }
}
```

Use this to verify candidate resolution, lifecycle, profiles, conversion, events, or proxy presence. A test that merely checks `notNull` is weak unless wiring is the actual risk.

Transaction tests must observe committed reality

Spring TestContext can run a test in a transaction that rolls back by default. That keeps data isolated but can hide production behavior:

- code under test may join the test transaction rather than create its normal boundary;
- flush/commit constraints may not appear until the test ends;
- lazy access remains possible because the test transaction is open;
- an after-commit listener will not run if the test rolls back.

For commit semantics, end/commit the test transaction deliberately or invoke through production boundaries and verify from a separate transaction/connection. Test both commit and rollback.

Preemptive timeout trap

Spring binds test-managed transaction state to the current thread. A preemptive test timeout that runs the test body on another thread can execute database work outside that transaction, so changes may commit despite expected rollback. Prefer non-preemptive time assertions or explicit cleanup for integration tests.

Context caching

The TestContext framework caches contexts by a key derived from configuration classes, profiles, property sources, customizers, and other inputs. Reusing the same configuration speeds a suite. Excess unique mocks/properties fragment the cache.

`@DirtiesContext` evicts a context and should be rare. Prefer resetting the mutated state you own. Spring Framework 7 can pause inactive cached contexts, but that does not excuse tests that leak threads or global state.

Proxy contract tests

JAVA

```java
@Test
void transactionalFailureRollsBackBothWrites() {
    assertThrows(RuntimeException.class,
            () -> service.place(failingCommand));

    assertEquals(0, countOrdersInNewTransaction());
    assertEquals(0, countReservationsInNewTransaction());
}
```

Call the injected service reference, not a manually constructed target. Assert durable outcomes. For AOP, include a matching call, a non-matching call, exception path, and advice order.

MVC testing boundary

Use mock request infrastructure to test routing, conversion, validation, status, headers, JSON shape, and exception translation without starting a real server. Use an actual server or deployed test only for servlet container/network/TLS behavior. Spring Boot test slices are Boot features and belong in **SD 05**.

Test doubles

- **Fake:** working simplified implementation, useful for domain contracts.
- **Stub:** returns predefined values.
- **Mock:** verifies interactions; use where interaction itself is the contract.
- **Spy:** wraps real behavior; can over-couple tests to implementation.

Do not mock value objects or every internal method. Verify observable state and boundary interaction.

External resources

Use a real compatible database/broker/containerized service when vendor behavior matters. H2 is useful for small transaction demonstrations but cannot prove MySQL plans, locking, types, or SQL dialect. Keep fixtures deterministic, migrations real, and failure diagnostics accessible.

WATCH OUT | Common mistakes

- Starting a full context for every domain unit test.
- Calling a target directly in a proxy behavior test.
- Trusting rollback-only tests to prove commit behavior.
- Hiding lazy-load bugs with a test transaction.
- Excessive `@DirtiesContext` and unique context configurations.
- Using H2 as proof of MySQL-specific semantics.

Interview angle

Interviewer: How do you test `@Transactional`?

Strong answer: I load a focused context with the real transaction manager and database-compatible resource, invoke the proxied application service, force a qualifying failure after the first write, then inspect durable state from a separate transaction. I also test successful commit, checked/unchecked rollback rules where customized, and self-invocation or propagation risks. A test that itself rolls back is not enough evidence.

Quick check

1. When is a pure unit test sufficient?
2. What does the TestContext cache key depend on?
3. How can a test transaction hide lazy-loading defects?
4. Why are preemptive timeouts dangerous with transactional tests?
5. Which database should prove vendor-specific locking?

Predict and debug

Predict: An after-commit listener is tested inside a rollback-by-default test transaction. It may never execute.

Debug: The suite creates hundreds of contexts. Compare profiles, properties, mocks, and configuration keys; consolidate stable fixtures and remove unnecessary context dirtiness.

Practice

- **Foundation:** Unit-test a constructor-injected service without Spring.
- **Foundation:** Write a focused context wiring test.
- **Interview Core:** Verify a proxy advice match and non-match.
- **Interview Core:** Prove rollback using state observed outside the failed transaction.
- **SDE-2 Follow-up:** Reduce a slow suite's context count while preserving isolation and production fidelity.

TRY IT | Readiness checkpoint

Continue when each test names the risk it proves and uses the smallest environment that can faithfully expose that risk.

SDE-2 CORE CHAPTER

Chapter 19 - Architecture, Extension Points, and Production Diagnostics

SDE-2 Spring skill is not measured by annotation count. It is the ability to keep the object graph understandable, extend the container safely, and diagnose runtime behavior from evidence.

A healthy application shape

TEXT

```
HTTP / message / schedule adapters
             |
             v
application use-case services  <- transaction and authorization boundary
             |
             v
domain policies and state      <- ordinary Java
             |
             v
ports -> database / remote / broker adapters
```

Spring assembles these layers. Domain code need not depend on `ApplicationContext`, `@Transactional`, HTTP types, or persistence entities when the separation adds value. Do not turn this into ceremony: a small application can use fewer layers while preserving explicit boundaries.

Container extension point selection

Need	Extension point
change/register bean metadata before instances	`BeanFactoryPostProcessor` or registry-level processor
wrap/inspect instances around initialization	`BeanPostProcessor`
create an object through a specialized factory contract	`FactoryBean<T>`
alter import/registration from annotation metadata	registrar/selector infrastructure
customize scope	`Scope` implementation
react to context lifecycle	listener or lifecycle interface

These are library and platform tools, not everyday application defaults. A custom extension must document phase, ordering, eligible beans, side effects, thread safety, native/AOT implications, and failure diagnostics.

FactoryBean versus factory-method bean

`@Bean PaymentClient paymentClient()` is an ordinary factory method that registers its returned product. `FactoryBean<PaymentClient>` is itself a special bean whose normal lookup returns the product; prefixing the name with `&` addresses the factory. Many candidates confuse the two.

Use `FactoryBean` when reusable factory semantics require access to the container contract. For application configuration, an `@Bean` method is usually simpler.

Avoid container reach-through

Weak:

JAVA

```java
final class PricingService implements ApplicationContextAware {
    private ApplicationContext context;

    Money calculate(String strategyName, Cart cart) {
        return context.getBean(strategyName, PricingStrategy.class).price(cart);
    }
}
```

Stronger:

JAVA

```java
final class PricingService {
    private final Map<PricingTier, PricingStrategy> strategies;

    PricingService(List<PricingStrategy> candidates) {
        this.strategies = candidates.stream().collect(Collectors.toUnmodifiableMap(
                PricingStrategy::tier, Function.identity()));
    }
}
```

The stronger class exposes its dependency and owns allowlisted business routing without bean-name coupling.

Startup diagnostics

For a failed context:

1. Record the active profiles and non-secret configuration sources.
2. Read the deepest cause and identify the injection point or factory method.
3. Determine whether failure is definition, resolution, construction, initialization, or post-processing.
4. List matching candidate names/types and why each was included.
5. Inspect conditions/profiles/imports and accidental broad scans.
6. Check early-creation and proxy-eligibility warnings.
7. Reproduce with the smallest context.

Do not "fix" startup by making required beans lazy. That moves the same defect to the first request.

Runtime proxy diagnostics

For missing transactions, metrics, authorization, caching, or async behavior:

- confirm managed bean and actual injected reference;
- inspect proxy type and advisors;
- verify external call versus self-invocation;
- check method visibility/finality and annotation resolution;
- confirm feature enablement and manager/executor selection;
- assert behavior with a failure path, not a proxy-class screenshot alone.

Performance model

Spring method interception overhead is rarely the first production bottleneck compared with database, network, serialization, lock wait, or unbounded work. Measure before replacing abstractions. More common framework-related costs include:

- very large contexts and broad scanning;
- blocking remote calls in startup or request threads;
- long transactions and pool exhaustion;
- unbounded async queues;
- accidental request-time bean creation;
- high-cardinality logs/metrics from generic aspects;
- fragmented test context caches.

Observability boundaries

For each use case record a stable operation name, duration, outcome category, trace correlation, and resource timings. Avoid raw customer/order IDs as metric labels. Logs should explain policy decisions without revealing secrets. Health and readiness should be bounded and should distinguish "process alive" from "safe to receive traffic." Full Spring Boot Actuator treatment is in **SD 05**.

Security posture

- Minimize package scanning and reflection over untrusted types.
- Do not evaluate user input as SpEL.
- Do not expose bean names, environment dumps, or stack traces publicly.
- Treat configuration values and event payloads as sensitive by classification.
- Keep authorization at explicit use-case boundaries; AOP may enforce it but tests must prove coverage.
- Patch supported Spring lines and review dependency advisories through the build and release process.

Spring Security, reactive WebFlux, Spring Batch, and Spring Integration receive dedicated treatment in **SD 10 - Spring Ecosystem Extensions**. This volume introduces boundaries without duplicating those books.

Incident scenario: wrong payment adapter

Symptom: production routes real payments to a sandbox endpoint after a deployment.

Evidence plan: inspect active profiles, effective non-secret endpoint, bean candidates, configuration import, and deployment change. Confirm the actual `PaymentGateway` bean class and settings. Do not dump tokens.

Correction: fail startup when the production deployment uses a non-approved host; replace broad profile combinations with one validated settings contract and explicit adapter selection; add deployment-policy and focused context tests.

WATCH OUT | Common mistakes

- Building a service locator on `ApplicationContext`.
- Writing custom post-processors for ordinary dependency injection.
- Making required infrastructure lazy to hide startup failures.
- Optimizing proxy overhead before measuring external work.
- Logging effective configuration including secrets.
- Treating health checks as unlimited integration tests.

Interview angle

Interviewer: How would you debug a Spring bean that exists but is not transactional?

Strong answer: I confirm the caller's reference is container-managed, inspect whether it is proxied and which advisors apply, verify the call crosses the proxy, check method visibility and resolved transaction metadata, confirm transaction management and the selected manager, then run a focused rollback test. I avoid toggling proxy mode blindly because the root cause may be object ownership, self-invocation, or exception rules.

Quick check

1. When is a custom post-processor justified?
2. How does `FactoryBean` differ from an `@Bean` method?
3. Why is making a missing dependency lazy not a repair?
4. Which costs usually dominate proxy overhead?
5. What must a readiness check avoid?

Practice

- **Foundation:** Place controller, use case, domain policy, port, and adapter in the architecture.
- **Foundation:** Match five needs to container extension points.
- **Interview Core:** Replace a context-based strategy lookup with explicit injection.
- **Interview Core:** Write a startup-failure diagnostic checklist for an ambiguous bean.
- **SDE-2 Follow-up:** Lead a wrong-adapter incident from containment through preventive controls.

TRY IT | Readiness checkpoint

Continue when you can extend Spring only at the correct lifecycle phase and diagnose wiring, proxy, transaction, and capacity incidents from a small evidence plan.

SDE-2 CORE CHAPTER

Chapter 20 - Realistic Spring Framework Interview Rounds

Answer each prompt aloud before reading the model response. A strong answer identifies object ownership, registration, resolution, proxy crossing, thread/transaction boundary, failure semantics, and proof.

Answer control loop

```text
1. Clarify the business invariant and caller.
2. Draw target, bean definition, context, and proxy.
3. Trace candidate resolution and lifecycle.
4. Mark thread, transaction manager, resource, and external calls.
5. Predict normal, failure, retry, and shutdown outcomes.
6. Choose the simplest explicit boundary.
7. Prove with focused tests and operational evidence.
```

1. Spring versus Spring Boot

Interviewer: What is the difference?

Strong answer: Spring Framework provides the IoC container, DI, AOP, transactions, events, validation, MVC, and testing foundations. Spring Boot builds on it with opinionated auto-configuration, starters, application bootstrap, external-configuration conventions, packaging, and operational integration. Boot does not replace Framework mechanics; it contributes bean definitions and configuration according to conditions.

2. IoC and DI

Interviewer: Explain IoC without saying "the framework controls everything."

Strong answer: A class declares collaborators through constructors or factory parameters, while a composition mechanism constructs and connects the object graph. Control of construction moves out of the business object. The class still owns its behavior and invariants. DI is the mechanism, and the Spring container is one implementation.

3. Bean creation trace

Interviewer: What happens when the context starts?

Strong answer: Configuration is parsed into bean definitions, definition processors run, eligible singletons are instantiated, dependencies are resolved and populated, lifecycle callbacks run through post-processors, post-processors may return proxies, and the context publishes refresh events. Lazy and non-singleton beans may be created later. Shutdown invokes eligible destruction callbacks.

4. Constructor or field injection

Interviewer: Why constructor injection?

Strong answer: It exposes required dependencies in the type contract, permits final references, fails incomplete graphs during startup, and enables pure Java tests. Field injection hides requirements and depends on reflective container mutation. Constructor injection does not make a class immutable or justify a constructor with fifteen dependencies.

5. Multiple implementations

Interviewer: Two gateways implement one interface. What happens?

Strong answer: An unqualified single injection point is ambiguous. If one is a real default I mark it primary; if selection is semantic I use a custom qualifier; if the use case needs all strategies I inject a collection and define ordering/routing explicitly. I do not rely on parameter names or add primary solely to silence startup.

6. Circular dependency

Interviewer: How do you fix `A -> B -> A`?

Strong answer: Constructor injection exposes the cycle. I look for shared logic to extract, orchestration to move above both services, a temporal event boundary, or responsibilities that should merge. `@Lazy` or self-injection can defer construction but usually hides the design issue, so I use them only with a justified lifecycle contract.

7. Singleton safety

Interviewer: Are Spring singleton beans thread-safe?

Strong answer: Not automatically. Singleton means one instance per bean definition per container, so request threads share it. Stateless services are typically safe; mutable fields require a concurrency design. I inspect collaborators and scoped proxies too. Request/session scope does not serialize access either.

8. Prototype in singleton

Interviewer: Why is the prototype reused?

Strong answer: The singleton's dependencies were resolved once during its construction, so it received one prototype instance. To request a new instance per operation I can inject `ObjectProvider`, use lookup/scoped proxy mechanisms, or simply construct a plain operation object if it has no container dependencies. I also own prototype cleanup.

9. @Configuration interception

Interviewer: Why does one `@Bean` method calling another return the singleton?

Strong answer: In full configuration mode, Spring subclass-enhances the configuration and intercepts factory calls through the container. Lite mode, including `proxyBeanMethods=false`, uses normal Java calls. I prefer factory method parameters, which make the dependency explicit and work in both modes.

10. Post-processor difference

Interviewer: `BeanFactoryPostProcessor` versus `BeanPostProcessor`?

Strong answer: The former changes or inspects bean-definition metadata before ordinary instances. The latter sees instances around initialization and can return a wrapper/proxy. Both are infrastructure; resolving application beans too early can make them miss later post-processing.

11. Event delivery

Interviewer: Is `publishEvent` asynchronous and durable?

Strong answer: Not by default. The default multicaster normally invokes listeners synchronously in the publishing thread, although it is configurable. Events are in-process and not durable. A transactional after-commit listener avoids pre-commit reaction but retains a crash window. Reliable cross-system delivery needs outbox/idempotency or a broker workflow.

12. Aspect does not run

Interviewer: An annotation is present, but advice is absent. Diagnose it.

Strong answer: I confirm the object came from the context, feature infrastructure is enabled, the caller has the proxy, the call is external rather than self-invocation, the method is visible/advisable for the proxy type, and the pointcut resolves against the actual method. Then I add a focused match/non-match test instead of changing annotations randomly.

13. JDK versus class proxy

Interviewer: Compare them.

Strong answer: A JDK proxy implements target interfaces and exposes those contracts. A class proxy subclasses the target and can expose class methods, but final classes/methods and private methods cannot be advised. Both are proxy boundaries; self-invocation remains a concern. I choose interfaces from design value, not solely for proxy mechanics.

14. Transaction placement

Interviewer: Controller, service, or repository?

Strong answer: Usually the externally invoked application-service method that owns one business invariant. Repository calls participate. The controller owns HTTP translation; repositories own persistence mechanics. I keep slow remote calls outside the database transaction and integration-test durable commit and rollback.

15. Checked exception

Interviewer: A checked exception leaves a transactional method. Rollback?

Strong answer: Under Spring's default declarative rules, ordinary checked exceptions do not trigger rollback, while unchecked exceptions and errors do. I configure `rollbackFor` when the checked exception means the unit failed, or translate the exception according to the boundary. I prove the result with committed-state tests.

16. Unexpected rollback

Interviewer: Why does the outer method get `UnexpectedRollbackException` after catching an inner error?

Strong answer: Both `REQUIRED` scopes joined one physical transaction. The inner failure marked it rollback-only. Catching the exception did not clear that status, so the outer commit could not succeed and Spring reported the unexpected rollback rather than returning false success.

17. REQUIRES_NEW

Interviewer: Use it for audit writes?

Strong answer: Only if audit durability must be independent of business rollback. It suspends the outer transaction, needs a separate resource, can exhaust the pool, and still requires a proxy crossing. Often an outbox or dedicated append-only audit path gives clearer durability and load behavior.

18. Async transaction

Interviewer: Does the caller's transaction reach `@Async` work?

Strong answer: Imperative transaction state is normally thread-bound, and async work executes on another executor thread. It does not inherit the caller transaction. The task can start a separate transaction. I pass immutable IDs/commands, observe exceptional completion, bound the executor, and use durable messaging if queued work cannot be lost.

19. Transactional test passed, production failed

Interviewer: Why?

Strong answer: A rollback-by-default test may keep the persistence context open, never prove commit-time constraints or after-commit listeners, and cause production code to join the test transaction. I add tests that invoke the proxy, commit where needed, and verify from a new transaction using the target database behavior.

20. Production pool exhaustion

Interviewer: Would you increase the pool?

Strong answer: First separate acquisition wait, transaction duration, database execution, lock wait, remote calls inside transactions, nested/new transactions, and leaks. A larger pool can overload the database. I shorten hold time and control concurrency, then size the pool from measured database capacity and service-level goals.

21. Validation and binding

Interviewer: Why not bind JSON directly to an entity?

Strong answer: It exposes writable fields, couples API to persistence, can trigger lazy behavior, and enables over-posting. I bind to a narrow request DTO, convert and structurally validate it, authorize, then load and mutate domain state explicitly. Database constraints still protect concurrent invariants.

22. Scheduled job in a cluster

Interviewer: How do you prevent duplicate work?

Strong answer: I assume every replica can trigger. I use a database lease/claim, partition ownership, leader scheduler, or external job system; process bounded idempotent units; make leases expire for recovery; and monitor lag, duplicates, and failed items. A local annotation is only a trigger.

23. Component scanning strategy

Interviewer: Why not scan the company root package?

Strong answer: It hides the registration boundary and can pull in unintended adapters, tests, or duplicate configuration. I use marker-type scans for application-owned components and explicit imports/bean methods for modules and infrastructure. The goal is a deterministic, reviewable graph.

24. Framework extension design

Interviewer: Your team wants a custom annotation that wraps every method. What do you ask?

Strong answer: I ask which policy and boundary it represents, exact pointcut, order relative to transactions/retries/security, self-invocation and proxy limits, allowed payload metadata, failure behavior, and how match/non-match paths are tested. If it hides domain workflow, I keep the behavior explicit instead.

Readiness signal

You are ready for Spring Boot when these answers start from the object graph and runtime boundary, not from a list of annotations.

SDE-2 CORE CHAPTER

Chapter 21 - Practice, Solutions, Readiness, and Sources

Attempt each group before reading the solution notes. The goal is executable reasoning, not memorized definitions.

Practice bank: 60 prompts

Foundation: 1-20

1 Define bean, bean definition, context, IoC, and DI.

2 Draw a plain Java graph for controller, service, and repository.

3 Register that graph with explicit `@Bean` methods.

4 Explain `BeanFactory` versus `ApplicationContext`.

5 Trace context refresh from definitions to exposed objects.

6 Compare constructor, setter, and field injection.

7 Resolve two implementations with a semantic qualifier.

8 Inject all validation strategies and define their order.

9 Explain why a missing component annotation is not always the registration bug.

10 Compare component scanning and explicit imports.

11 Build and validate a typed settings record.

12 Read a classpath resource packaged inside a JAR.

13 Explain singleton scope precisely.

14 Demonstrate prototype lookup twice.

15 Explain prototype injection into a singleton.

16 List lifecycle stages through destruction.

17 Compare definition and instance post-processors.

18 Publish one immutable application event.

19 Convert a string into a domain identifier.

20 Separate structural, domain, authorization, and database validation.

Interview Core: 21-45

21 Remove a constructor circular dependency without `@Lazy`.

22 Explain full and lite `@Configuration` behavior.

23 Prove one factory method parameter receives the managed singleton.

24 Decide whether a profile or typed property should select an adapter.

25 Diagnose a resource that works from the IDE but not a JAR.

26 Show why a mutable singleton counter races.

27 Inject a request-scoped collaborator into a singleton safely.

28 Explain why request context does not follow an async task.

29 Compare command, application event, and durable message.

30 Design an after-commit reaction and name its crash window.

31 Define join point, pointcut, advice, aspect, target, and proxy.

32 Test an aspect match, non-match, and exception path.

33 Order authorization, retry, transaction, and metrics advice.

34 Explain JDK versus subclass proxies.

35 Refactor an advised self-invocation.

36 Diagnose advice on a final or private method.

37 Place one transaction around order and inventory writes.

38 Predict rollback for checked and unchecked exceptions.

39 Explain rollback-only and unexpected rollback.

40 Compare `REQUIRED`, `REQUIRES_NEW`, and `NESTED`.

41 Explain why `readOnly` is not authorization.

42 Move an HTTP call out of a transaction.

43 Map a Spring MVC request from filter to response.

44 Design safe validation and problem responses.

45 Write a focused proxy-backed rollback integration test.

SDE-2 Follow-up: 46-60

46 Classify deadlock, lock timeout, duplicate key, and uncertain commit for retry.

47 Order retry and transaction advice and prove the order.

48 Design request idempotency with a payload fingerprint.

49 Model unknown remote payment outcome and reconciliation.

50 Design an outbox relay with lease, retry, idempotency, and metrics.

51 Capacity-plan `REQUIRES_NEW` under concurrent outer transactions.

52 Design a bounded executor and overload policy from downstream capacity.

53 Coordinate one scheduled logical job across ten replicas.

54 Diagnose a growing async queue with idle application CPU.

55 Reduce a test suite with hundreds of context cache keys.

56 Prove after-commit listener behavior in a test.

57 Diagnose an unproxied transactional bean from runtime evidence.

58 Recover from a wrong-profile/wrong-adapter production deployment.

59 Design a privacy-safe cross-cutting telemetry aspect.

60 Review a custom container extension for lifecycle and AOT risks.

TRY IT | Debugging exercises: 15 code reviews

1. A service field is annotated `@Autowired` and cannot be constructed in a unit test. Refactor it.
2. Two `Clock` beans exist and an unqualified constructor parameter fails startup. State two valid repairs.
3. A broad scan registers both fake and production payment gateways. Narrow the graph.
4. A singleton stores `currentCustomerId` in a field. Explain the race and redesign it.
5. A prototype connection wrapper expects `@PreDestroy`. Define ownership.
6. A lite configuration calls another `@Bean` method directly. Remove duplicate construction.
7. A listener sends email before the transaction later rolls back. Select an appropriate reaction design.
8. Around advice does not call `proceed()`. Predict the result.
9. `this.reserve()` has `@Transactional(REQUIRES_NEW)` but joins the caller. Explain why.
10. A checked import exception commits partial rows. Repair rollback semantics.
11. An inner `REQUIRED` failure is caught and outer commit throws unexpectedly. Trace it.
12. An async method accepts a managed entity and later throws lazy-loading errors. Redesign the command.
13. Six replicas run the same cleanup schedule. Add coordination.
14. A transactional test passes because lazy state remains open. Reproduce the production boundary.
15. An error aspect logs passwords and tokens. Replace its telemetry contract.

Solution notes

Foundation solutions

1-5 should distinguish metadata, instance, container, and proxy; use the context as the composition root; and describe refresh without claiming every lazy/prototype bean is created. 6-10 should make required dependencies explicit, qualify semantics rather than names, treat order as a contract, and constrain scanning. 11-15 should convert properties once, stream resources, define singleton per bean/container, and explain prototype resolution and cleanup. 16-20 should separate lifecycle phases, use events as in-process facts, convert before validation, and preserve database constraints for concurrent invariants.

Interview Core solutions

21-25 should redesign cycles, use parameter injection across configuration modes, keep environment selection coarse, and avoid filesystem assumptions. 26-30 should identify shared mutable state, use scoped indirection, treat thread context explicitly, and use outbox for durable delivery. 31-36 should draw proxy dispatch, make pointcuts and order testable, and move policy boundaries to externally invoked visible methods. 37-42 should center the business invariant, state rollback rules, distinguish logical and physical transactions, and shorten resource hold time. 43-45 should keep DTO/status/error contracts explicit and verify durable rollback through the injected proxy.

SDE-2 solutions

46-50 require categorized failures, fresh-transaction bounded retries, request/message identifiers, uncertain-outcome state, and a crash-recoverable outbox. 51-55 require measured resource demand, bounded queues, distributed job ownership, bottleneck evidence, and context-key consolidation. 56-60 should deliberately commit when testing after-commit behavior, inspect actual advisors/call paths, validate deployment configuration, redact high-risk data, and document extension lifecycle/order/compatibility.

Debugging solutions

1-5 favor constructor injection, explicit selection, narrow scanning, immutable method arguments, and caller-owned prototype cleanup. 6-10 use factory parameters, transaction-aware event phases or outbox, exactly one `proceed`, external proxy crossings, and type-safe rollback rules. 11-15 preserve rollback-only truth, pass immutable IDs to async work, coordinate cluster jobs, test closed transaction boundaries, and allowlist telemetry fields.

Five cumulative assessments

Assessment 1: object graph

Build an order context with explicit modules, two gateway candidates, typed configuration, lifecycle, and a pure Java service test.

Pass: every bean has one registration path; no field injection, hidden service lookup, broad scan, ambiguous candidate, or secret default.

Assessment 2: proxy reasoning

Add timing and audit advice, choose proxy mode, demonstrate one self-invocation failure, refactor it, and prove advisor order.

Pass: match/non-match, exception, method visibility, runtime type, and external call path are all explained.

Assessment 3: transaction and delivery

Place an order and inventory reservation, add idempotency, publish an outbox record, and handle deadlock plus uncertain payment outcome.

Pass: local invariant is atomic, retries start fresh transactions, remote calls do not hold locks, and duplicates are safe.

Assessment 4: HTTP and concurrency

Design create/find endpoints, validation, problem responses, async notification, and a clustered cleanup job.

Pass: DTOs are narrow, statuses precise, executor bounded, context propagation explicit, and scheduling coordinated.

Assessment 5: test and incident round

Write pure unit, context, proxy, transaction-commit, MVC, and external-resource tests; diagnose a wrong adapter plus pool exhaustion.

Pass: each test proves a named risk, committed state is observed, evidence precedes capacity changes, and no secret is exposed.

Final readiness assessment

You are ready for **SD 05 - Spring Boot** when you can, without notes:

- construct and explain a context from plain Java through managed beans;
- predict candidate resolution, lifecycle, scope, and configuration mode;
- draw JDK/subclass proxy and self-invocation paths;
- place transactions from invariants and trace rollback/propagation;
- distinguish in-process events, async tasks, and durable messaging;
- map MVC requests through validation, service, and safe response;
- choose a test level and prove committed/failed outcomes;
- diagnose startup, proxy, pool, and scheduled-work incidents from evidence.

Official sources and version baseline

The prose and examples were audited against primary Spring documentation. These links are also the upgrade path when behavior changes:

- Spring Framework reference: https://docs.spring.io/spring-framework/reference/
- IoC container and beans: https://docs.spring.io/spring-framework/reference/core/beans.html
- Bean scopes: https://docs.spring.io/spring-framework/reference/core/beans/factory-scopes.html
- Container extension points: https://docs.spring.io/spring-framework/reference/core/beans/factory-extension.html
- Spring AOP proxy mechanics: https://docs.spring.io/spring-framework/reference/core/aop/proxying.html
- Transaction management: https://docs.spring.io/spring-framework/reference/data-access/transaction.html
- Transaction propagation: https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative/tx-propagation.html
- Task execution and scheduling: https://docs.spring.io/spring-framework/reference/integration/scheduling.html
- Spring MVC reference: https://docs.spring.io/spring-framework/reference/web/webmvc.html
- TestContext framework: https://docs.spring.io/spring-framework/reference/testing/testcontext-framework.html
- Supported Spring Framework lines: https://github.com/spring-projects/spring-framework/wiki/Spring-Framework-Versions
- Spring Framework 7.0 release notes: https://github.com/spring-projects/spring-framework/wiki/Spring-Framework-7.0-Release-Notes

Version baseline for executable labs: Java 21, Spring Framework 7.0.8, JUnit Jupiter 5.11.4, and H2 2.3.232. H2 proves local Spring transaction mechanics only; it does not validate MySQL-specific locking or query behavior.

Cross-book boundaries

- Spring Boot auto-configuration, Actuator, packaging, and Boot testing: **SD 05**.
- Repository derivation and Spring Data abstractions: **SD 06**.
- MySQL isolation, indexes, locks, and recovery: **SD 02**.
- Hibernate/JPA lifecycle and fetch behavior: **SD 03**.
- Spring Security, WebFlux, Batch, Integration, and specialized testing: **SD 10**.
- Kafka delivery and Spring Kafka: **SD 09**.

The core rule remains simple: first draw the Java objects and runtime boundary; then choose the Spring mechanism that makes that boundary explicit and testable.

SDE-2 CORE CHAPTER

Chapter 22 - Java 21 Spring Runtime Companion

This dependency-free executable model validates bean-graph creation order and cycles, qualifier/primary candidate resolution, external proxy versus self-invocation rules, and default/custom transaction rollback policy; the Maven fixture separately executes real Spring behavior.

JAVA (continued 1/7)

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Queue;
import java.util.Set;

/**
 * Dependency-free Java 21 companion for Spring Framework interview reasoning.
 *
 * <p>This is not a replacement container. It makes three hidden framework
 * decisions explicit: dependency order, proxy interception, and rollback.
 */
public final class SpringFrameworkInterviewCompanion {
    private SpringFrameworkInterviewCompanion() {
    }

    public static void main(String[] args) {
        validatesDependencyOrderAndCycles();
        validatesCandidateResolution();
        validatesProxyCrossingRules();
        validatesRollbackRules();
        System.out.println("SpringFrameworkInterviewCompanion checks passed");
    }

    private static void validatesDependencyOrderAndCycles() {
        Map<String, Set<String>> graph = new LinkedHashMap<>();
        graph.put("orderController", Set.of("orderService"));
        graph.put("orderService", Set.of("orderRepository", "clock"));
```

JAVA (continued 2/7)

```
        graph.put("orderRepository", Set.of("dataSource"));
        graph.put("clock", Set.of());
        graph.put("dataSource", Set.of());

        List<String> order = creationOrder(graph);
        require(order.indexOf("dataSource") < order.indexOf("orderRepository"),
                "data source must precede repository");
        require(order.indexOf("orderRepository") < order.indexOf("orderService"),
                "repository must precede service");
        require(order.indexOf("orderService") < order.indexOf("orderController"),
                "service must precede controller");

        Map<String, Set<String>> cycle = Map.of(
                "orderService", Set.of("pricingService"),
                "pricingService", Set.of("orderService"));
        expectFailure(() -> creationOrder(cycle), "cycle");
    }

    private static void validatesCandidateResolution() {
        List<Candidate> gateways = List.of(
                new Candidate("stripe", Set.of("online"), true),
                new Candidate("offline", Set.of("offline"), false));

        require(resolve(gateways, "offline").name().equals("offline"),
                "qualifier must win");
        require(resolve(gateways, null).name().equals("stripe"),
                "single primary must win without qualifier");

        expectFailure(() -> resolve(List.of(
                new Candidate("first", Set.of(), false),
                new Candidate("second", Set.of(), false)), null), "ambiguous");
    }
```

JAVA (continued 3/7)

```
    private static void validatesProxyCrossingRules() {
        MethodShape publicMethod = new MethodShape(true, false, false);
        require(intercepted(CallPath.EXTERNAL_PROXY, ProxyKind.SUBCLASS, publicMethod),
                "external public subclass-proxy call should be intercepted");
        require(!intercepted(CallPath.SELF_INVOCATION, ProxyKind.SUBCLASS, publicMethod),
                "self invocation must bypass proxy advice");
        require(!intercepted(CallPath.EXTERNAL_PROXY, ProxyKind.SUBCLASS,
                        new MethodShape(true, true, false)),
                "final method cannot be overridden by subclass proxy");
        require(!intercepted(CallPath.EXTERNAL_PROXY, ProxyKind.SUBCLASS,
                        new MethodShape(false, false, true)),
                "private method cannot be advised by subclass proxy");
        require(intercepted(CallPath.EXTERNAL_PROXY, ProxyKind.JDK_INTERFACE, publicMethod),
                "interface method reached through JDK proxy should be intercepted");
    }

    private static void validatesRollbackRules() {
        require(outcome(new IllegalStateException("failed"), Set.of())
                        == TransactionOutcome.ROLLBACK,
                "unchecked exception should roll back by default");
        require(outcome(new ImportCheckedException("failed"), Set.of())
                        == TransactionOutcome.COMMIT,
                "checked exception should commit under default rules");
        require(outcome(new ImportCheckedException("failed"),
                        Set.of(ImportCheckedException.class))
                        == TransactionOutcome.ROLLBACK,
                "explicit checked rollback rule should roll back");
    }

    static List<String> creationOrder(Map<String, Set<String>> dependencies) {
        Objects.requireNonNull(dependencies, "dependencies");
        Map<String, Integer> remaining = new HashMap<>();
        Map<String, List<String>> dependents = new HashMap<>();
```

JAVA (continued 4/7)

```
        for (Map.Entry<String, Set<String>> entry : dependencies.entrySet()) {
            remaining.put(entry.getKey(), entry.getValue().size());
            for (String dependency : entry.getValue()) {
                if (!dependencies.containsKey(dependency)) {
                    throw new IllegalArgumentException(
                            "missing dependency " + dependency + " for " + entry.getKey());
                }
                dependents.computeIfAbsent(dependency, ignored -> new ArrayList<>())
                        .add(entry.getKey());
            }
        }

        Queue<String> ready = new ArrayDeque<>();
        dependencies.keySet().stream()
                .filter(name -> remaining.get(name) == 0)
                .sorted()
                .forEach(ready::add);

        List<String> result = new ArrayList<>();
        while (!ready.isEmpty()) {
            String created = ready.remove();
            result.add(created);
            List<String> next = new ArrayList<>(
                    dependents.getOrDefault(created, List.of()));
            next.sort(String::compareTo);
            for (String dependent : next) {
                int count = remaining.compute(dependent, (key, value) -> value - 1);
                if (count == 0) {
                    ready.add(dependent);
                }
            }
        }
```

JAVA (continued 5/7)

```
        if (result.size() != dependencies.size()) {
            throw new IllegalArgumentException("dependency cycle detected");
        }
        return List.copyOf(result);
    }

    static Candidate resolve(Collection<Candidate> candidates, String qualifier) {
        List<Candidate> matches = candidates.stream()
                .filter(candidate -> qualifier == null
                        || candidate.qualifiers().contains(qualifier))
                .toList();
        if (matches.size() == 1) {
            return matches.getFirst();
        }
        if (qualifier != null) {
            throw new IllegalArgumentException(
                    matches.isEmpty() ? "no qualified candidate" : "ambiguous qualifier");
        }
        List<Candidate> primary = matches.stream().filter(Candidate::primary).toList();
        if (primary.size() == 1) {
            return primary.getFirst();
        }
        throw new IllegalArgumentException(
                matches.isEmpty() ? "no candidate" : "ambiguous candidates");
    }

    static boolean intercepted(
            CallPath path, ProxyKind kind, MethodShape method) {
        if (path != CallPath.EXTERNAL_PROXY) {
            return false;
        }
        if (kind == ProxyKind.JDK_INTERFACE) {
```

JAVA (continued 6/7)

```
            return method.publiclyExposed();
        }
        return method.publiclyExposed() && !method.finalMethod() && !method.privateMethod();
    }

    static TransactionOutcome outcome(
            Throwable failure, Set<Class<? extends Throwable>> checkedRollbackRules) {
        if (failure instanceof RuntimeException || failure instanceof Error) {
            return TransactionOutcome.ROLLBACK;
        }
        boolean matched = checkedRollbackRules.stream()
                .anyMatch(type -> type.isAssignableFrom(failure.getClass()));
        return matched ? TransactionOutcome.ROLLBACK : TransactionOutcome.COMMIT;
    }

    private static void expectFailure(Runnable action, String expectedText) {
        try {
            action.run();
            throw new AssertionError("expected failure containing " + expectedText);
        } catch (IllegalArgumentException failure) {
            require(failure.getMessage().contains(expectedText),
                    "unexpected failure: " + failure.getMessage());
        }
    }

    private static void require(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    record Candidate(String name, Set<String> qualifiers, boolean primary) {
        Candidate {
```

JAVA (continued 7/7)

```
            Objects.requireNonNull(name, "name");
            qualifiers = Set.copyOf(qualifiers);
        }
    }

    record MethodShape(boolean publiclyExposed, boolean finalMethod, boolean privateMethod) {
    }

    enum CallPath {
        EXTERNAL_PROXY,
        SELF_INVOCATION,
        DIRECT_TARGET
    }

    enum ProxyKind {
        JDK_INTERFACE,
        SUBCLASS
    }

    enum TransactionOutcome {
        COMMIT,
        ROLLBACK
    }

    static final class ImportCheckedException extends Exception {
        private static final long serialVersionUID = 1L;

        ImportCheckedException(String message) {
            super(message);
        }
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: plain Java object graphs, first context, definitions, injection, configuration, lifecycle, and scopes
- Interview Core: events, validation, AOP, proxy mechanics, transactions, MVC, and focused tests
- SDE-2 Follow-up: propagation, retries, uncertainty, outbox, bounded executors, clustered jobs, extension points, and incidents
- Readiness: complete 24 answered interview rounds, 170 structured practice/debug tasks, five cumulative assessments, and both executable validation layers

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: SD 03 - Hibernate and JPA for Java Backend Interviews

Series index

Next book: SD 05 - Spring Boot for Java Backend Engineers

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: System Design and Backend - Book 04 of 14 - Study Step 19E. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.